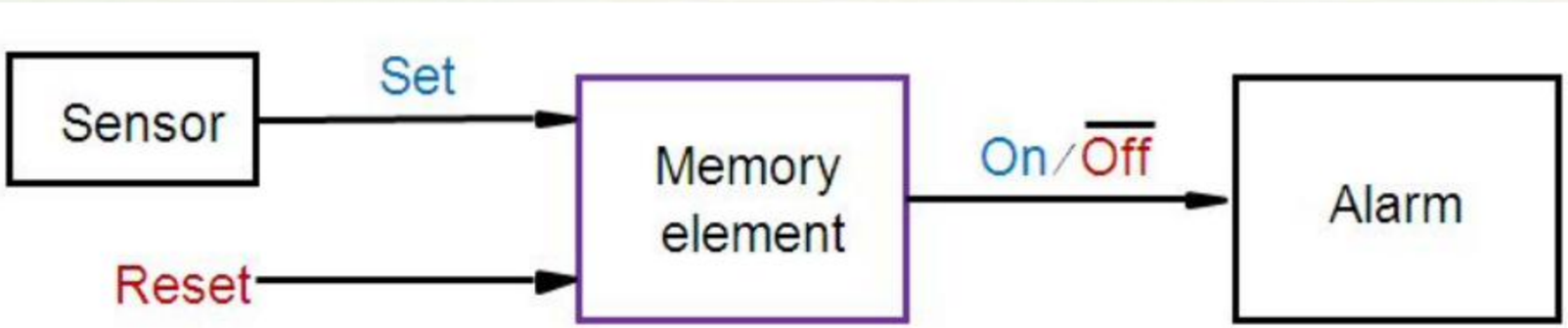
CH5 Basic Storage Units:

Latches, Flip-flops and Counters

5.1 A Brief Introduction

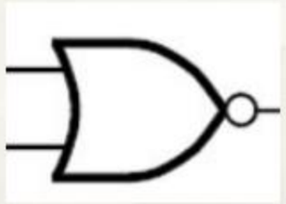
An alarm system



Control of an alarm system

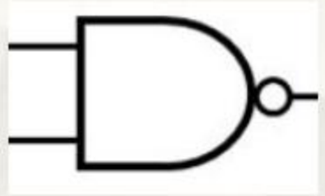
- *'Set'* triggers the alarm.
- The alarm remains active even if *Set* =0.
  - The memory element must store the state.
  - The present $On / \overline{Off}$ is determined by not only inputs but also the past state.
- The alarm is turned off if *Reset* = 1.

Review: NOR gate & NAND gate



x_1	x_2	f
0	0	1
0	1	0
1	0	0
1	1	0

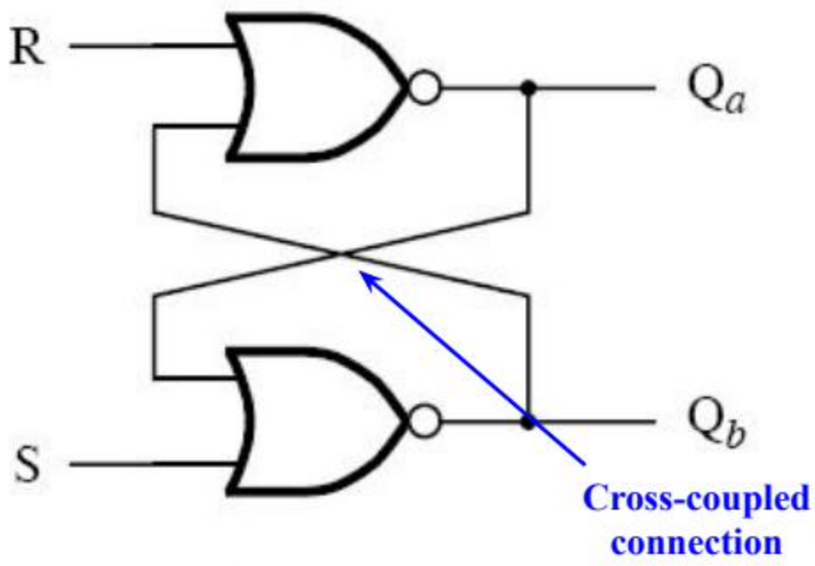
NOR's truth table



x_1	x_2	f
0	0	1
0	1	1
1	0	1
1	1	0

NAND's truth table

5.2 Basic Latch



(a) Circuit

Complements of each other in normal state

S	R	Q_a	Q_b	
0	0	0/1	1/0	(no change)
0	1	0	1	
1	0	1	0	
1	1	0	0	

Anomalous state

(b) Characteristic table

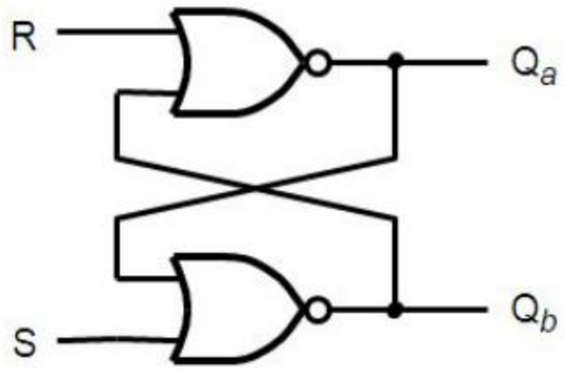
State stored!

When $S = 0$ & $R = 0$, $Q^{n+1} = Q^n$

Q^{n+1} (or $Q(t+1)$) : next state

Q^n (or $Q(t)$) : present state

More about the anomalous state



S	R	Q_a	Q_b	
0	0	0/1	1/0	(no change)
0	1	0	1	
1	0	1	0	
1	1	*d*	*d*	

- What if S and R go back to 0 simultaneously ?
  - Outputs Oscillate between 0 & 1
  - Although S and R do not change simultaneously, the next state is not easily predicted.
- Critical to avoid having both S & R =1.

Logic expression

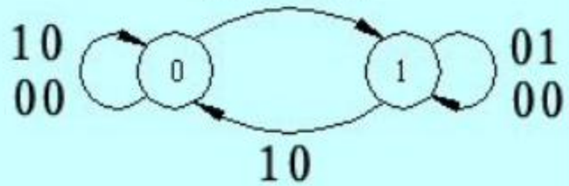
Q^n \ RS	00	01	11	10
0	0	1	d	0
1	1	1	d	0

$$Q^{n+1} = S + \overline{R}Q^n \qquad Constraint : RS = 0$$

State diagram

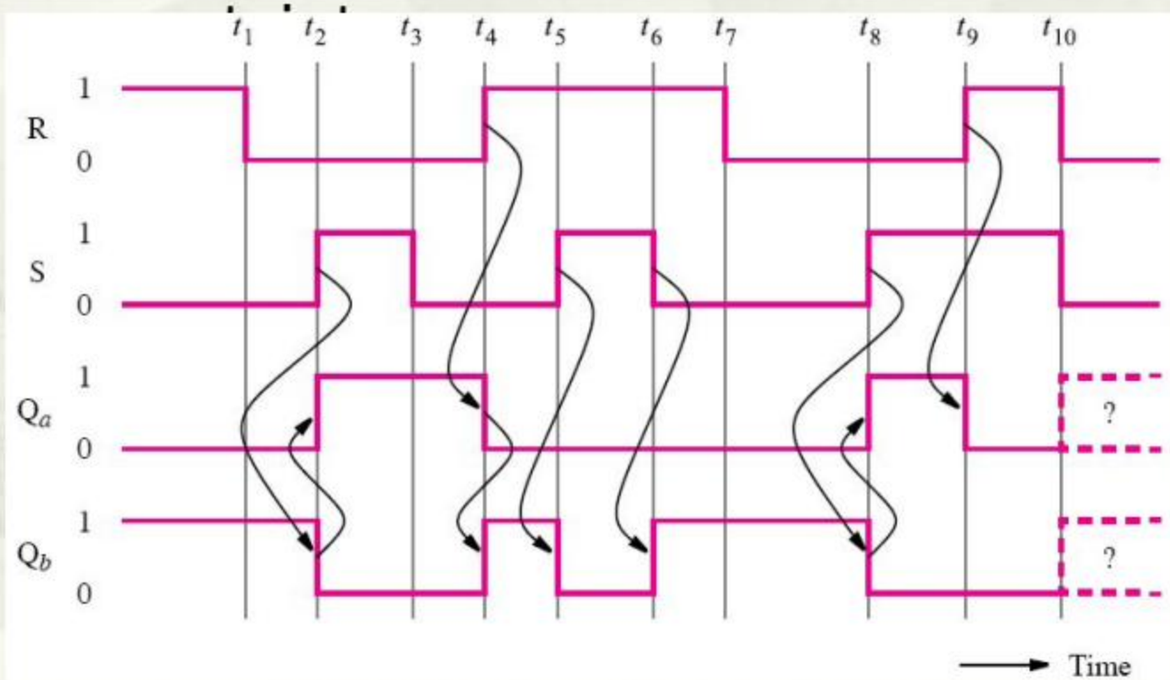
- Another type of characteristic table

Q^n \ RS	00	01	11	10
0	0	1	d	0
1	1	1	d	0

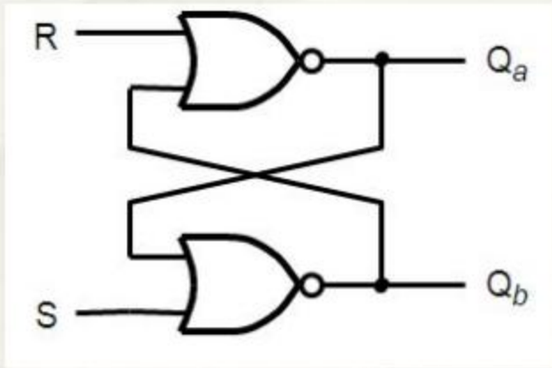


Timing diagram

- The basic latch changes its state once there's a change in one of input values which leads to unstability and



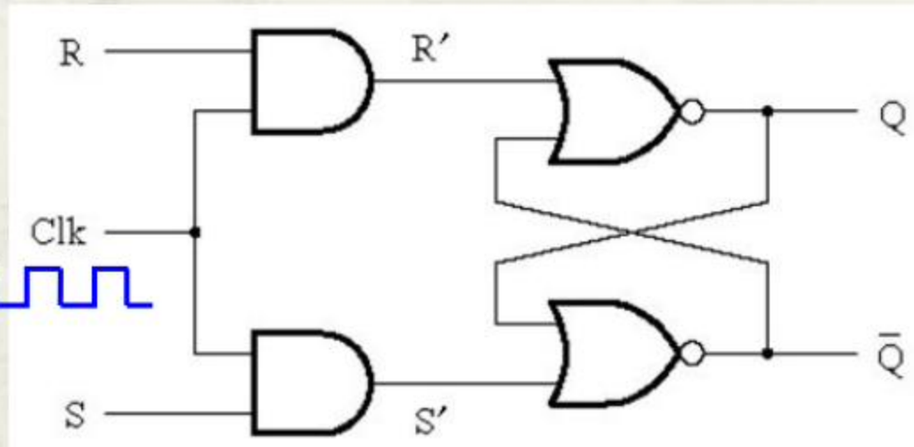
Irregularity of waveforms



S	R	Q_a	Q_b	
0	0	0/1	1/0	(no change)
0	1	0	1	
1	0	1	0	
1	1	0	0	

5.2 Gated SR Latch

- An input disturbance may affect the state for a basic latch.
  - State cannot be retained consistently.



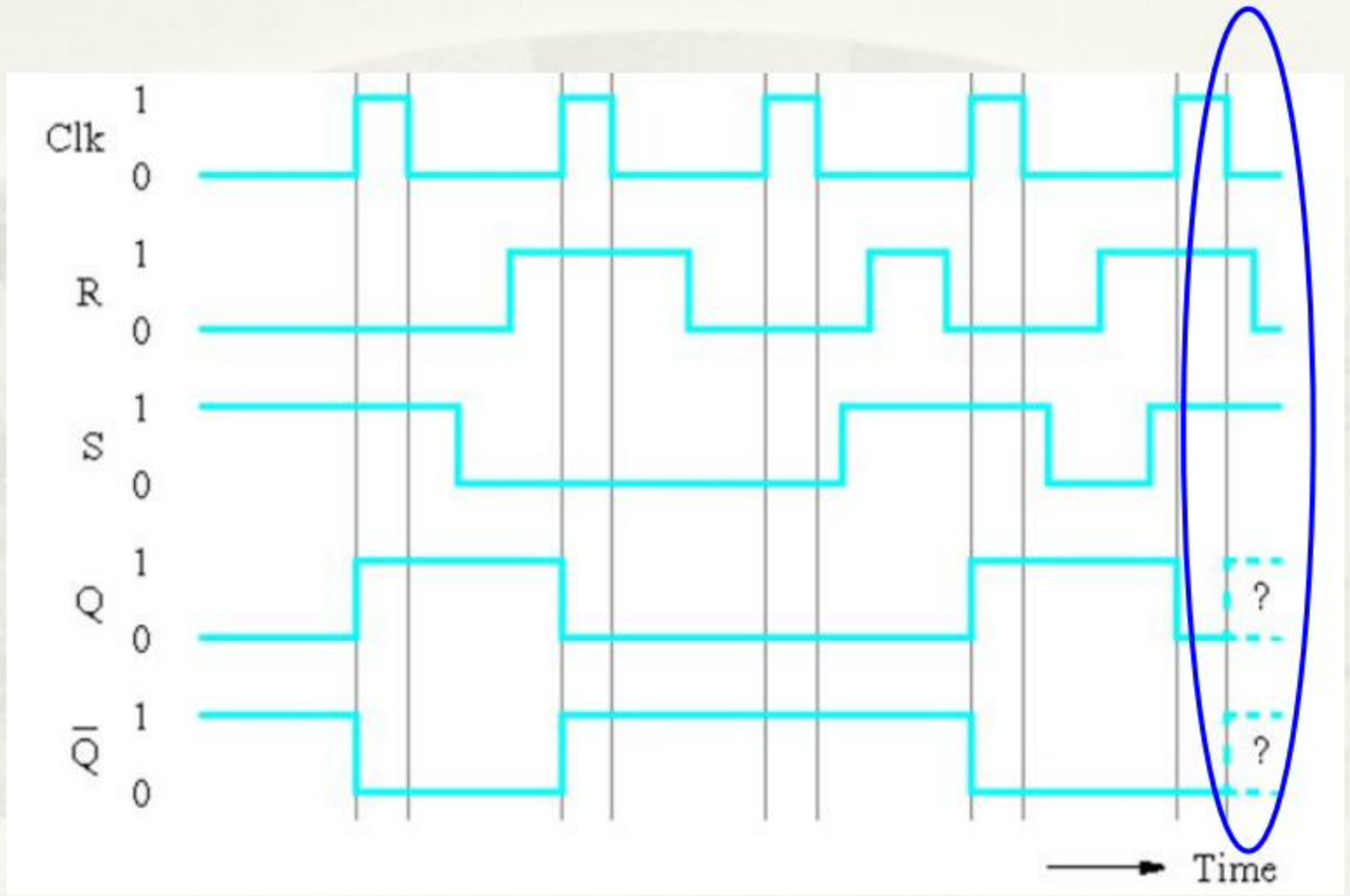
(a) Circuit

Clk	S	R	Q(t + 1)
0	x	x	Q(t) (no change)
1	0	0	Q(t) (no change)
1	0	1	0
1	1	0	1
1	1	1	x

(b) Characteristic table

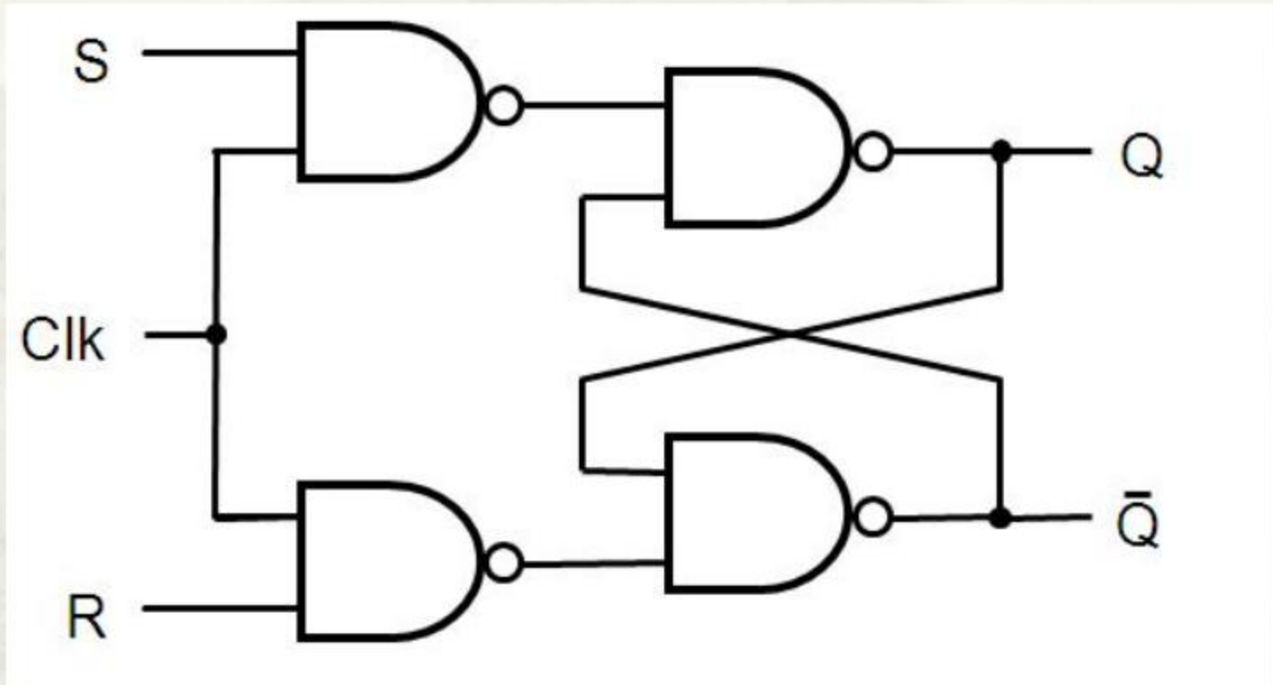
- The clock signal allows changes in states to occur at periodic time intervals.

Timing diagram of gated SR latch

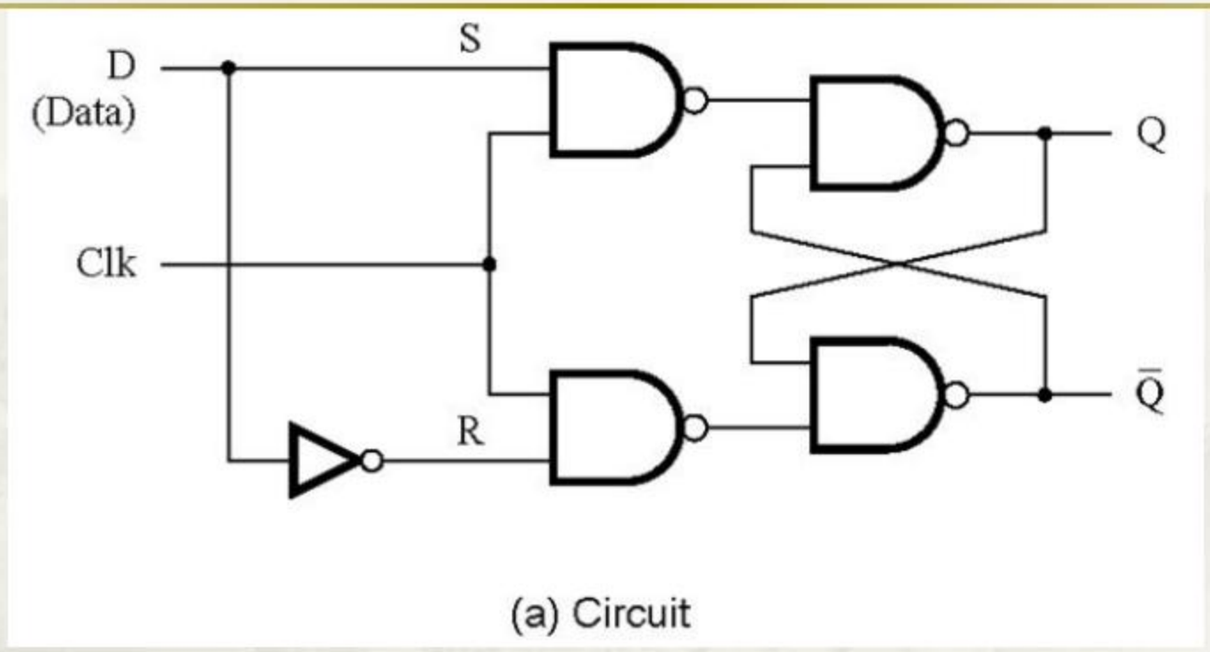


More regular output Q & Qc

Gated SR with NAND gates



5.3 Gated D Latch



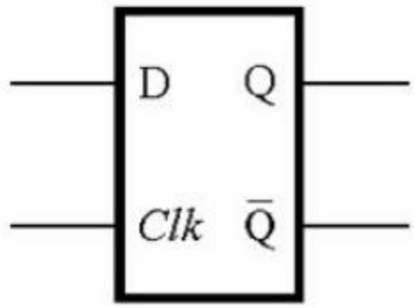
(a) Circuit

- A gated SR latch in essence.
- 1 input D
- S & R are always complements of each other.

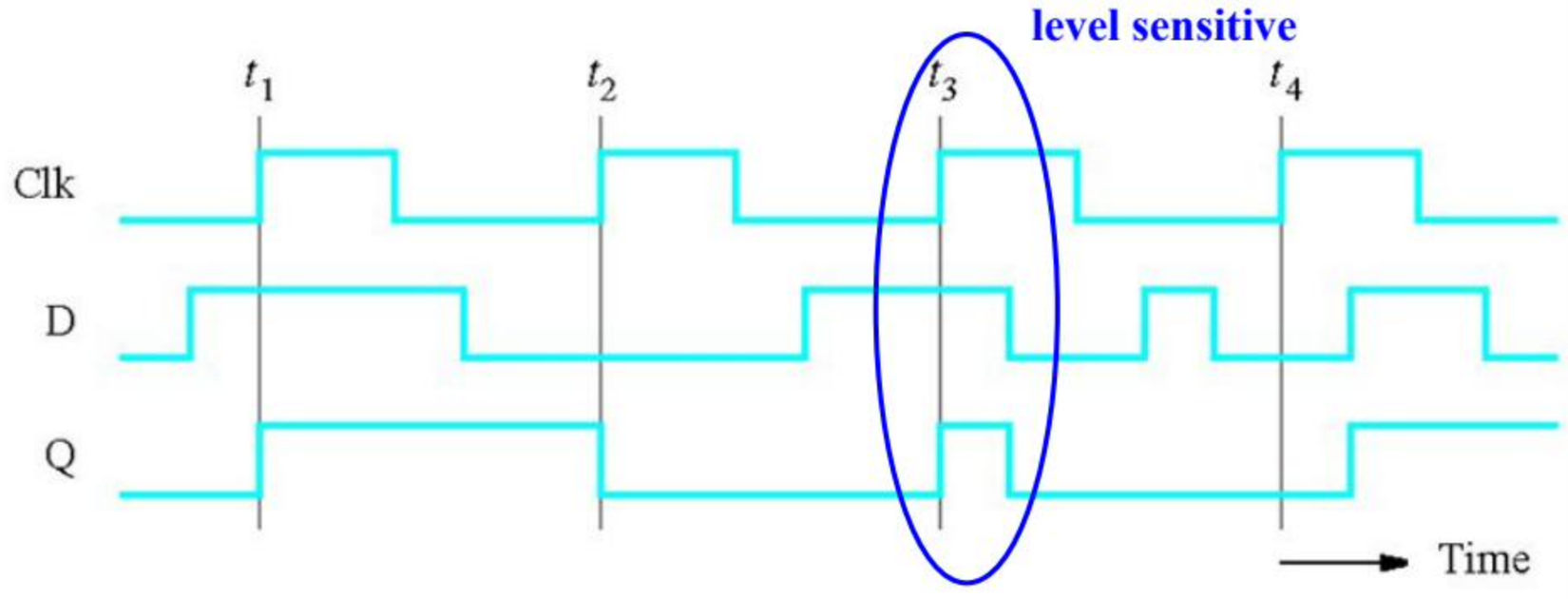
Clk	D	Q(t + 1)
0	x	Q(t)
1	0	0
1	1	1

$$Q^{n+1} = D$$

(b) Characteristic table



(c) Graphical symbol

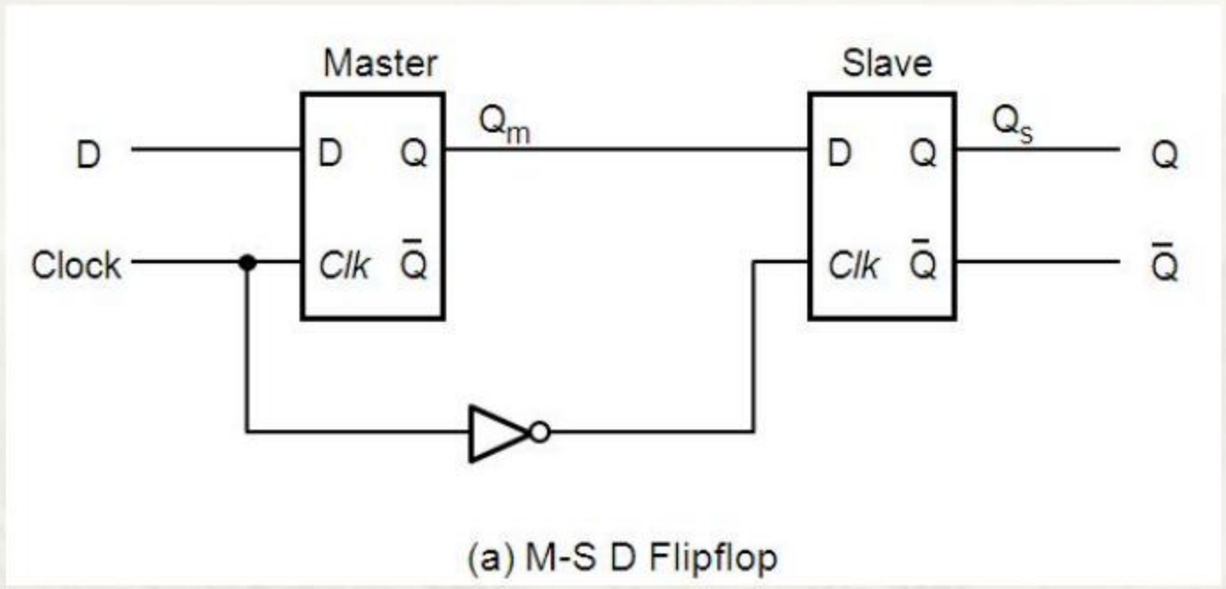


(d) Timing diagram

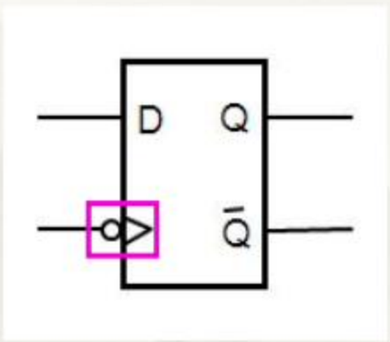
Basic SR, gated SR and gated D are all level sensitive!

5.4 Master-Slave and Edge-Triggered D

Flip-flop

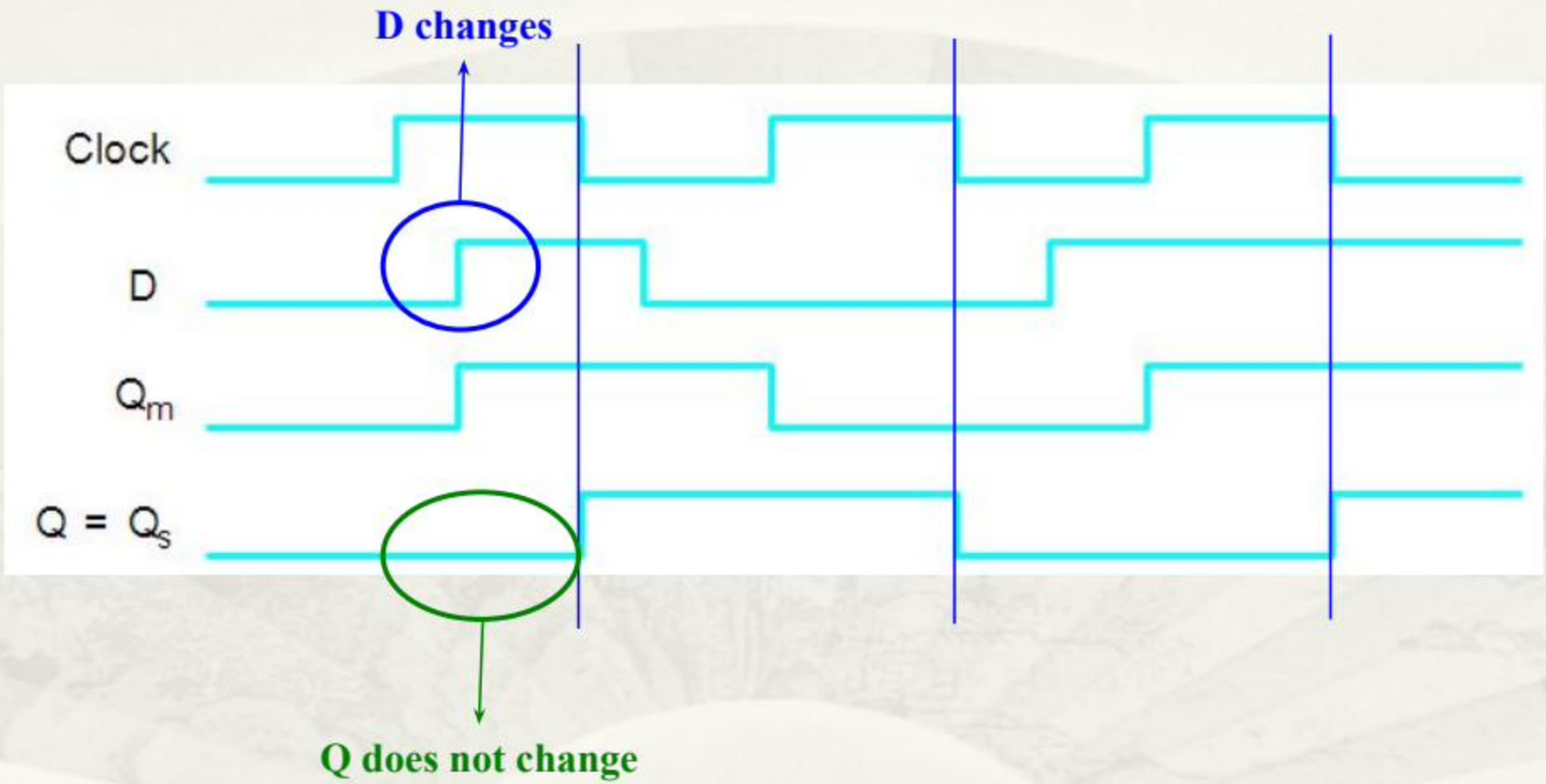


(a) M-S D Flipflop



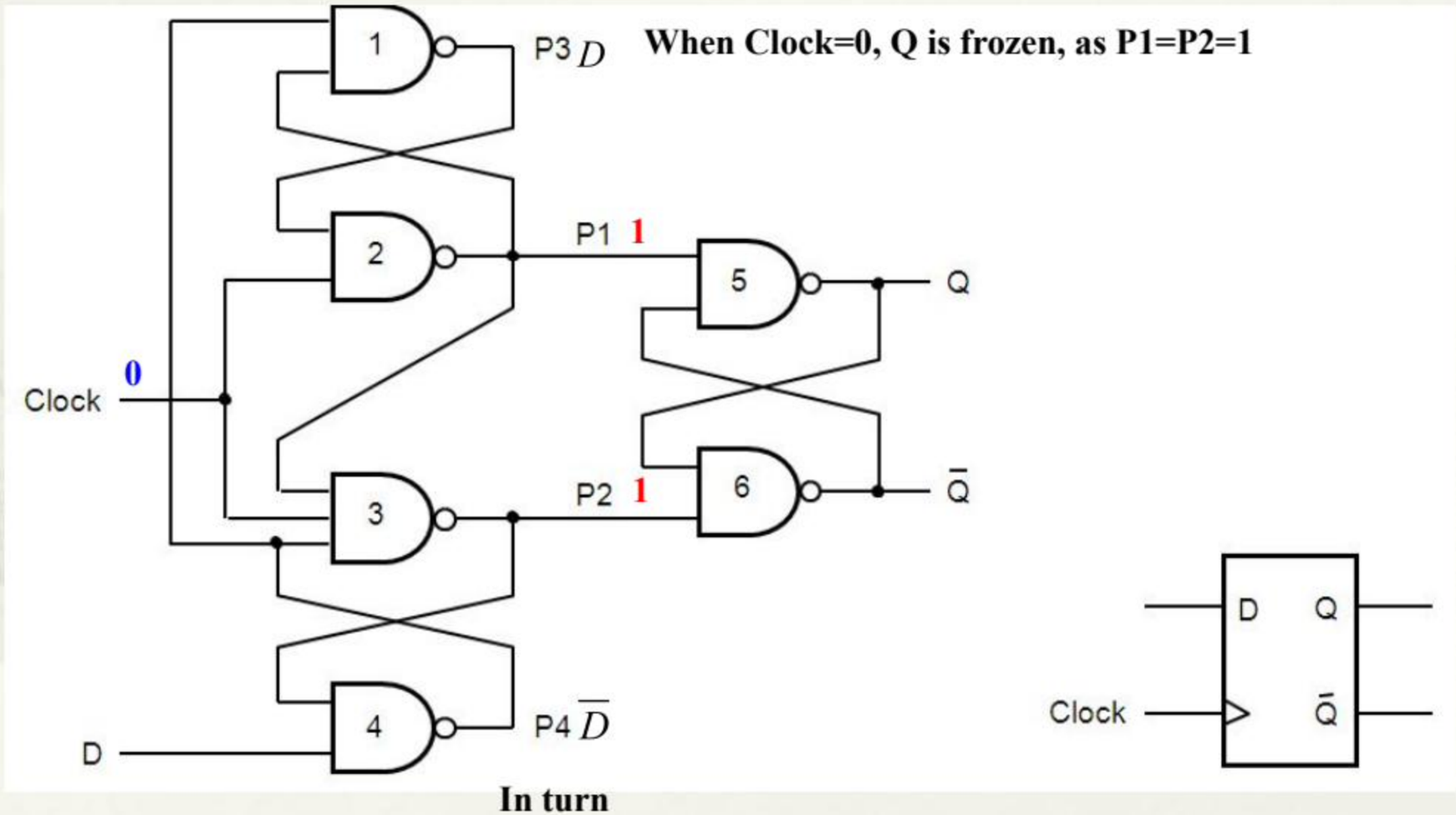
- Qm doesn't change when Clock=0
- Qs(Q) doesn't change when Clock=1
- Q changes only when Clock changes from 1 to 0(falling edge)
- **Flip-flop: an edge-triggered storage element that changes its output state only at the edge of Clock.**

Timing diagram



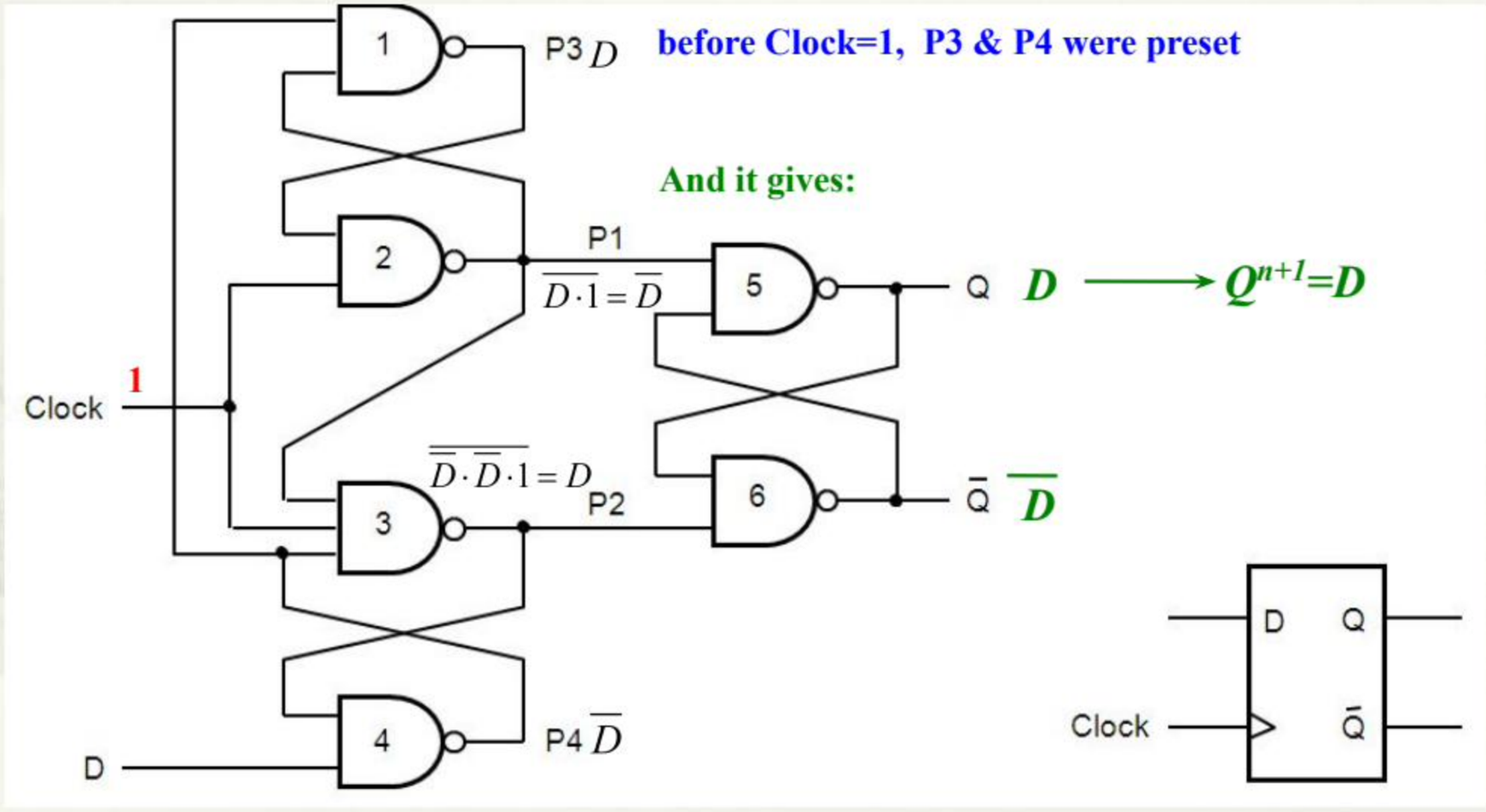
A different edge-triggered Dff

And then

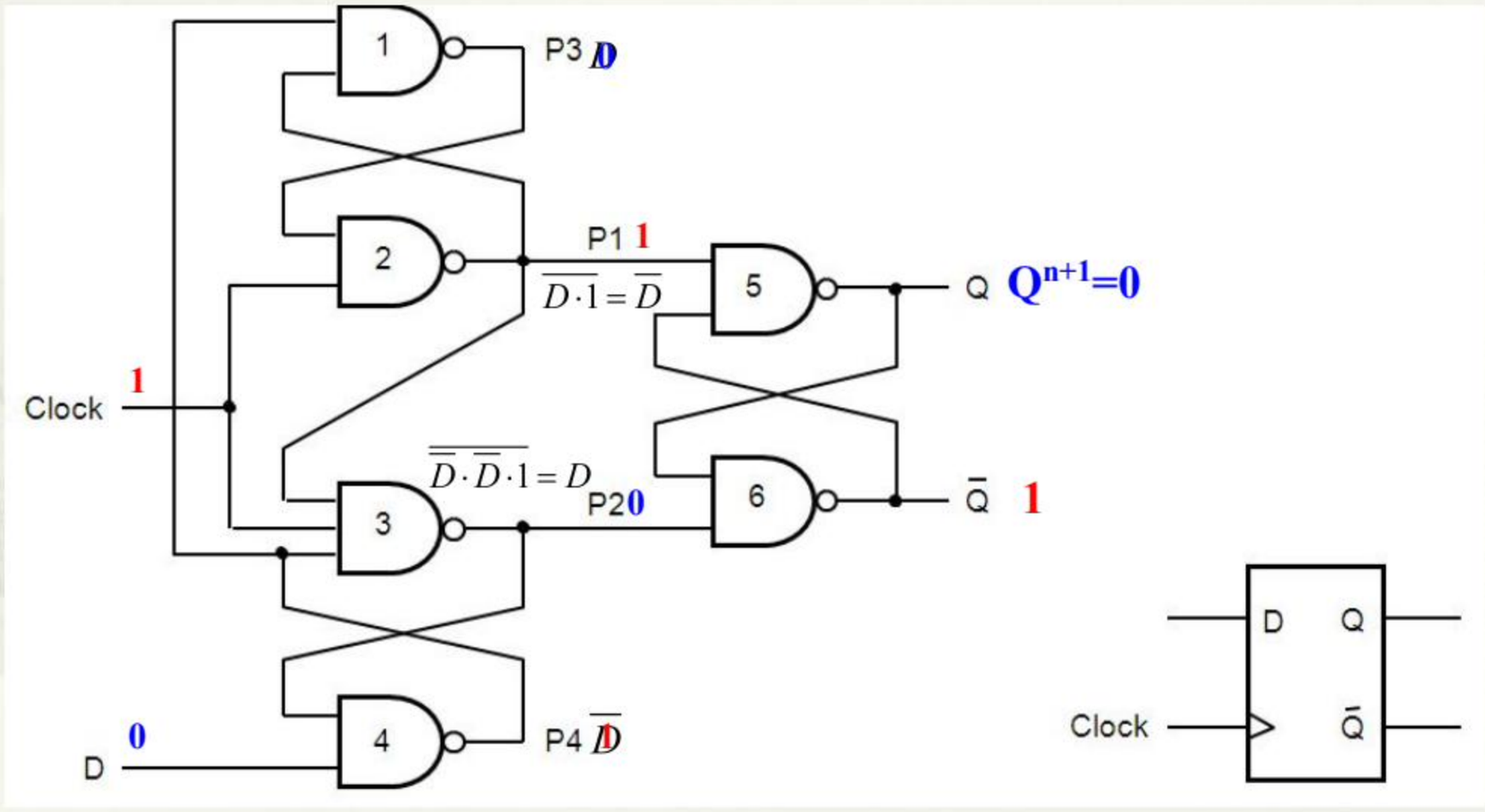


In turn

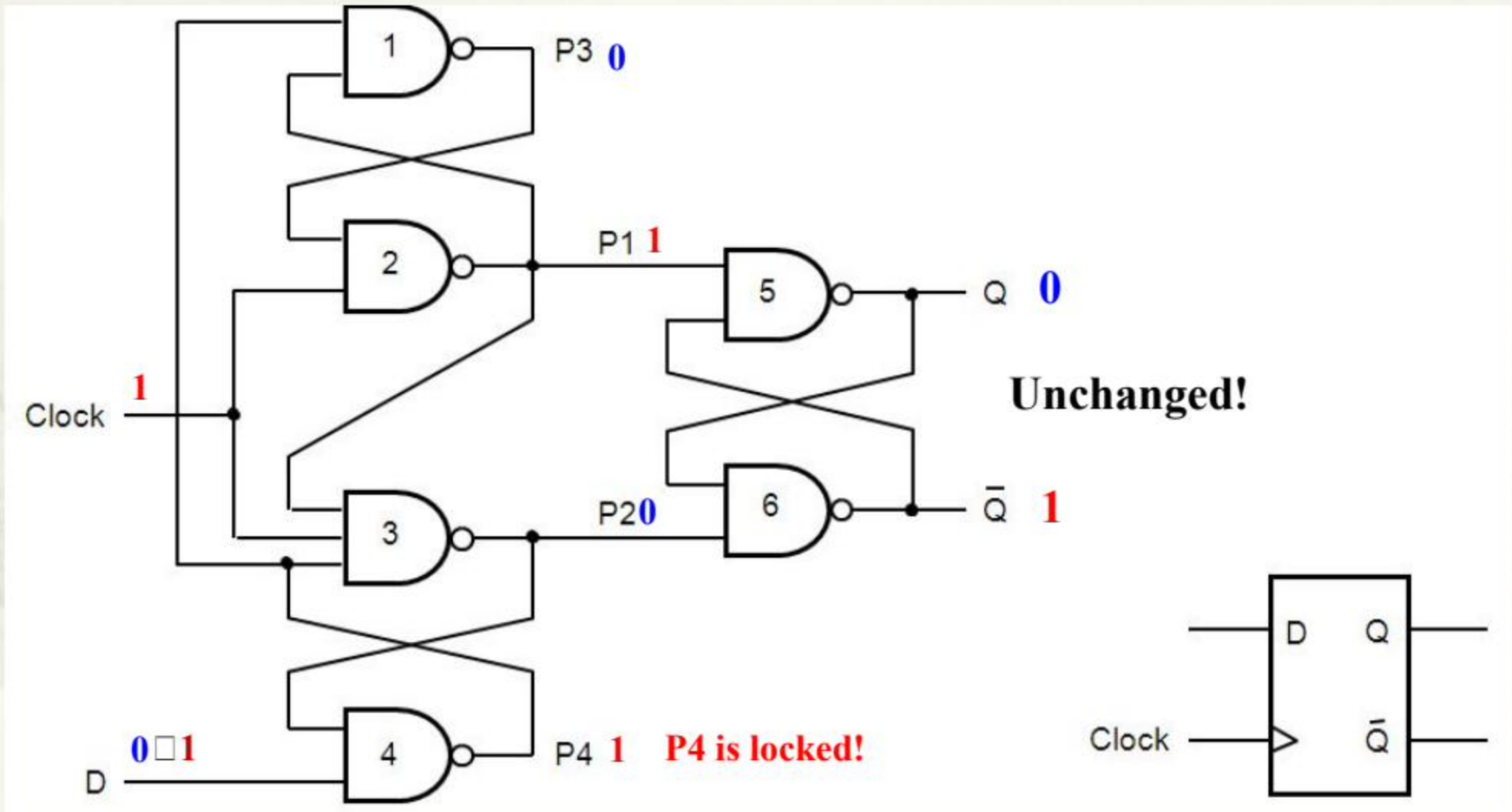
A different edge-triggered Dff



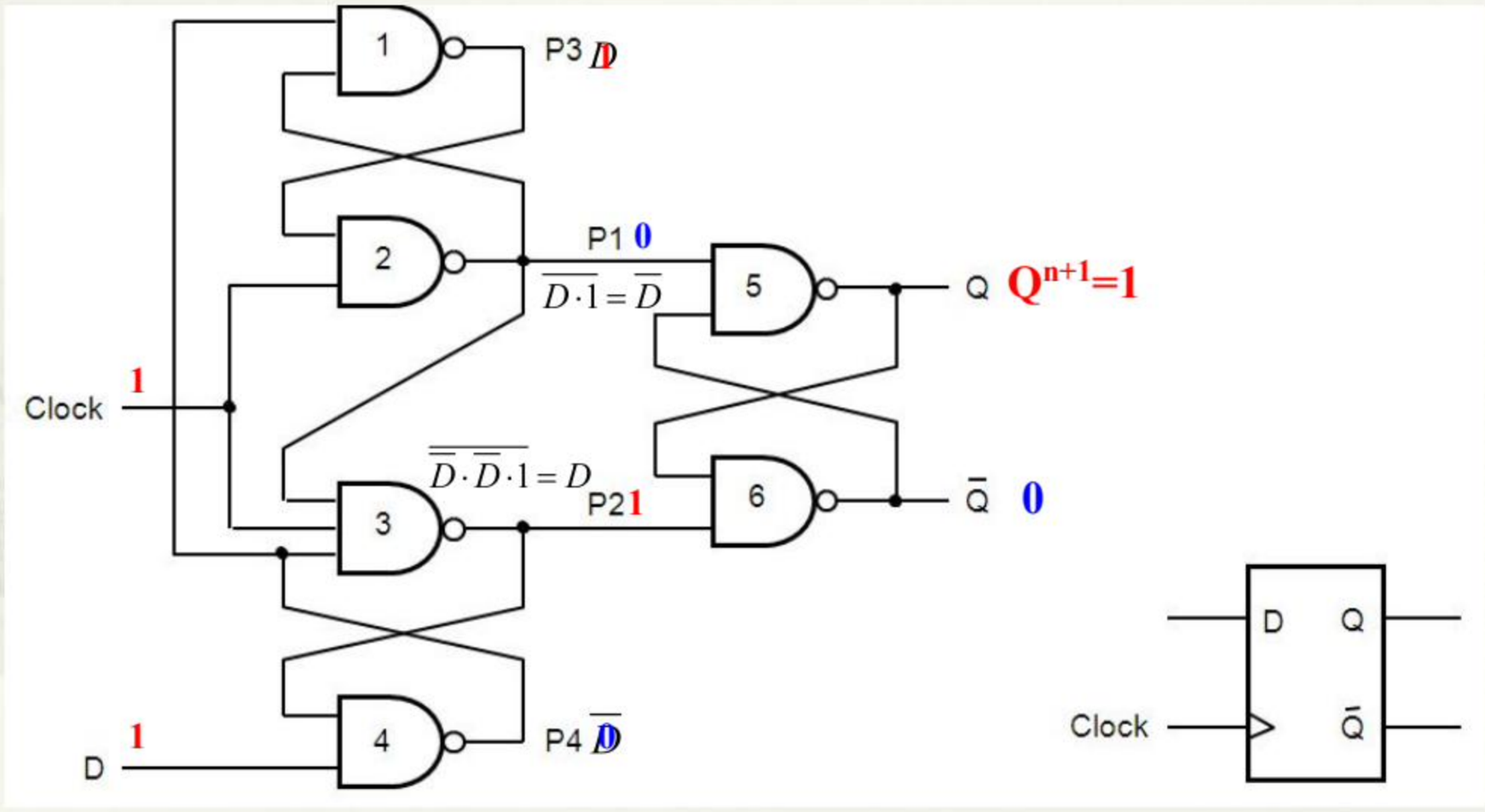
Suppose D=0 and D changes from 0□1 during Clk=1



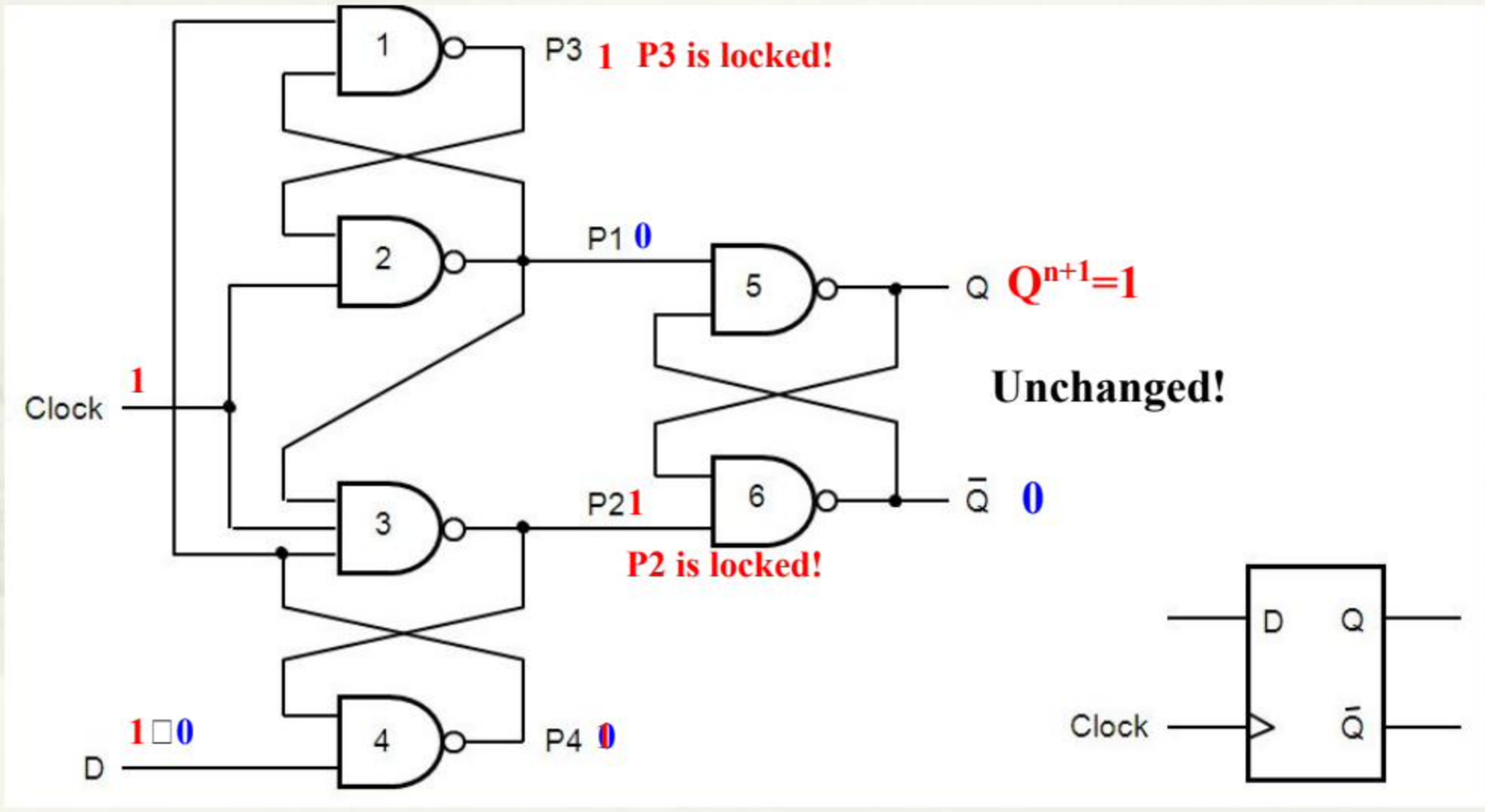
Suppose D=0 and D changes from 0□1 during Clk=1



Suppose D=1 and D changes from 1□0 during Clk=1

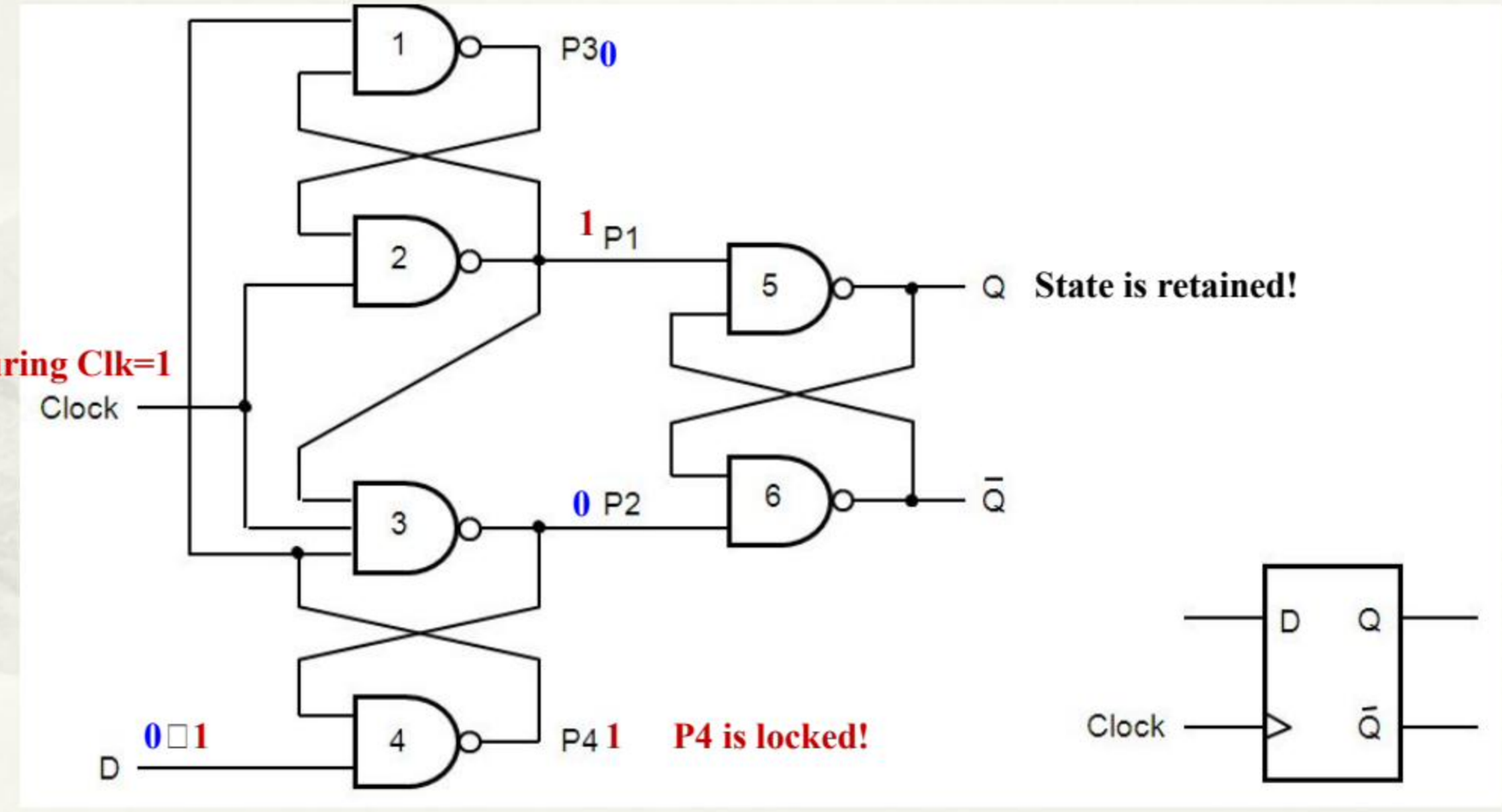


Suppose D=1 and D changes from 1□0 during Clk=1



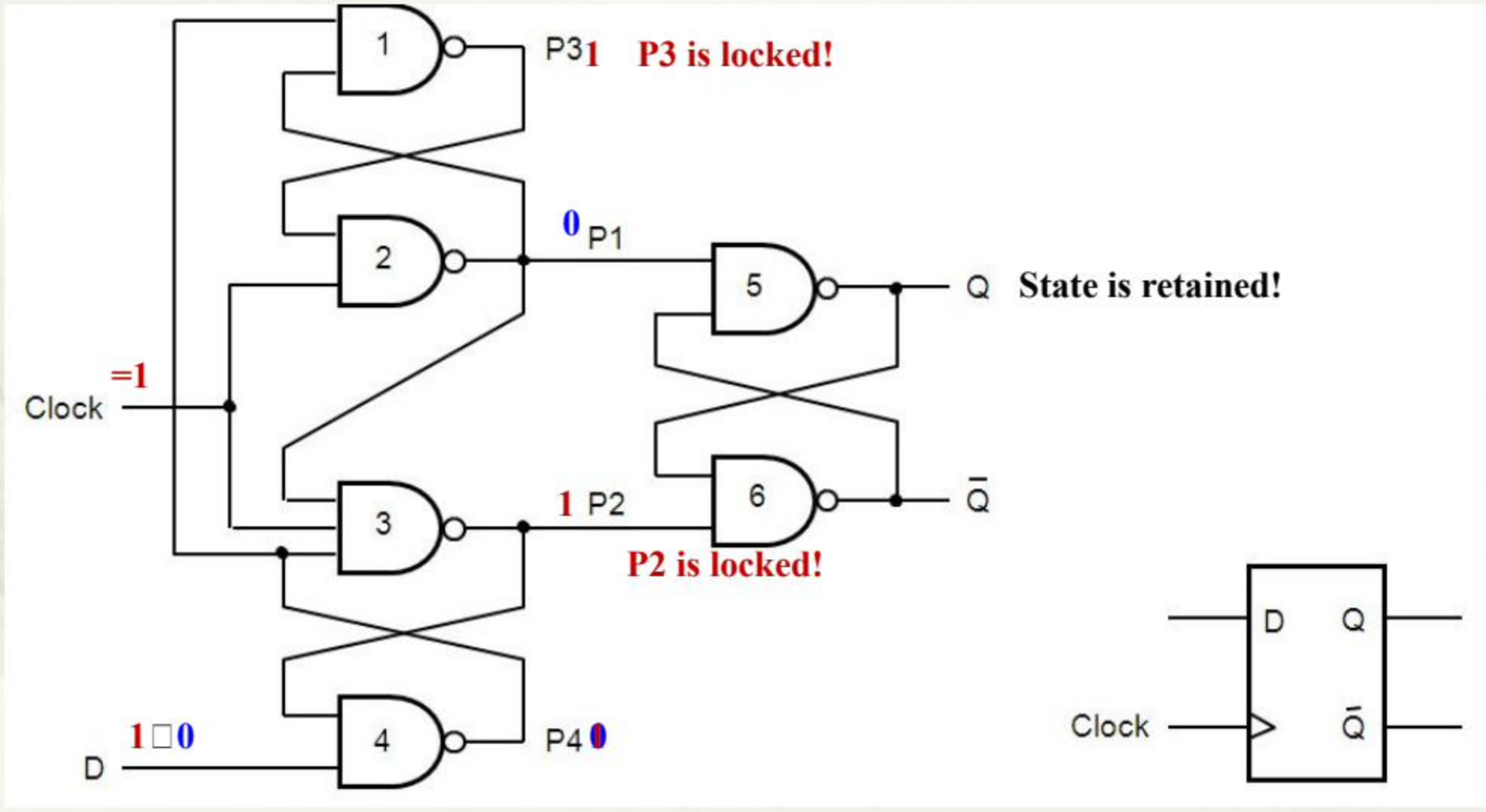
Q is retained regardless of changes of D! ⟶ **Edge triggered!**

A different edge-triggered Dff



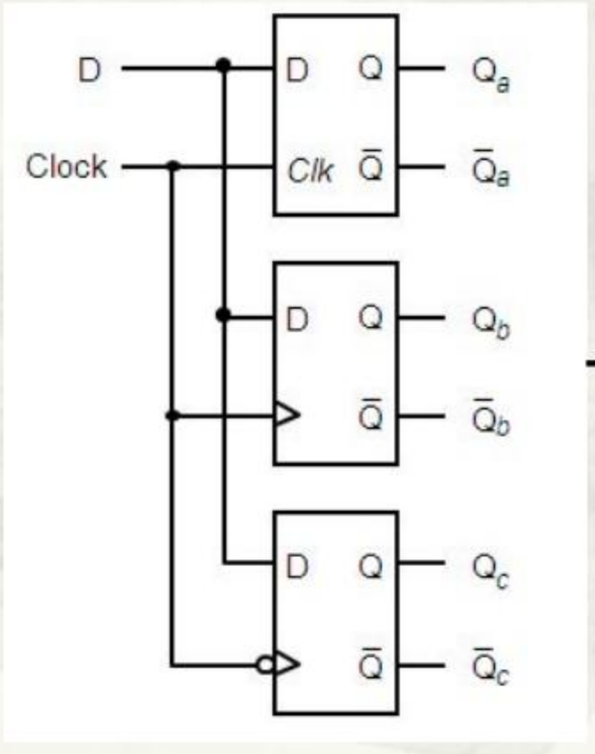
Edge-triggered principle

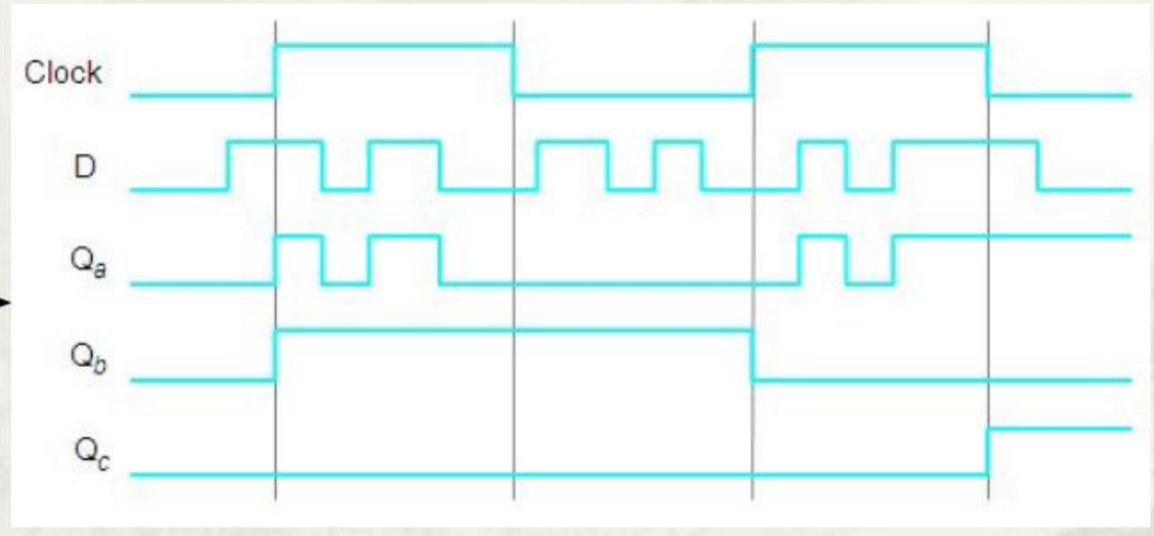
A different edge-triggered Dff



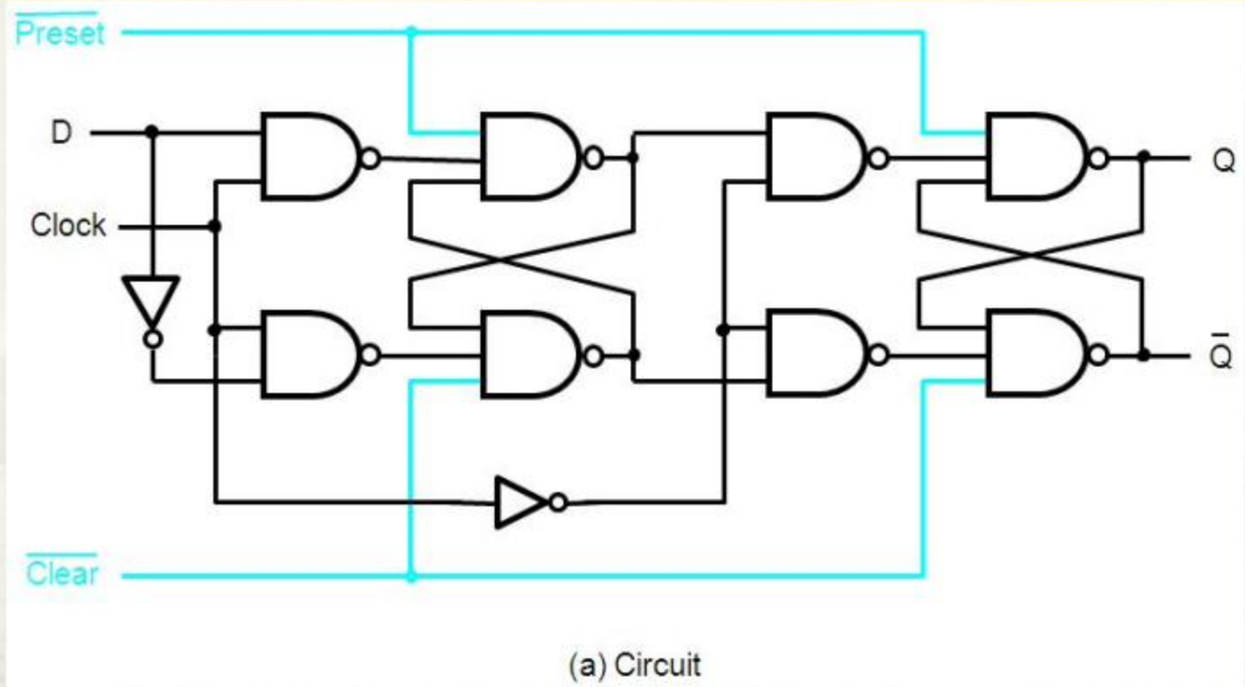
Edge-triggered principle

Level sensitive Vs. Edge-triggered storage element

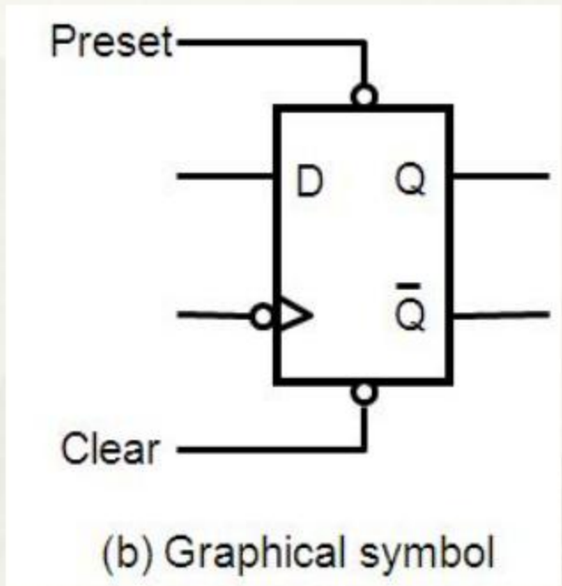




D flip-flop with Clear and Preset



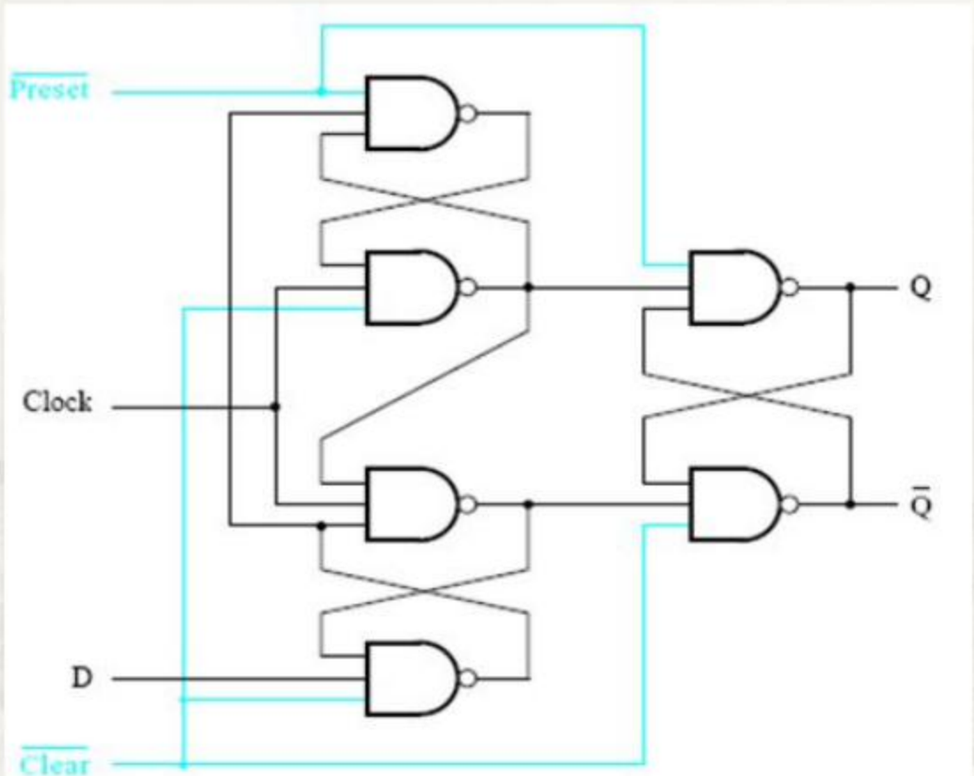
(a) Circuit



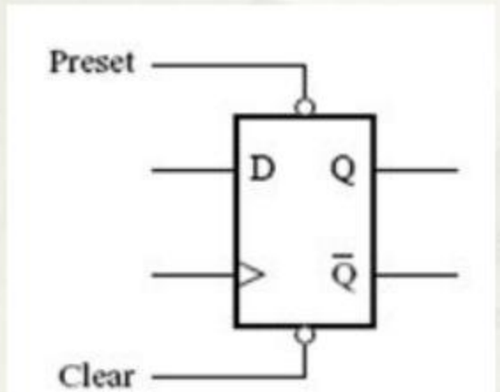
(b) Graphical symbol

- Active low.
- Asynchronous *Clear* and *Preset*.
- Falling-edge triggered

A rising-edge triggered Dff with Clear & Preset

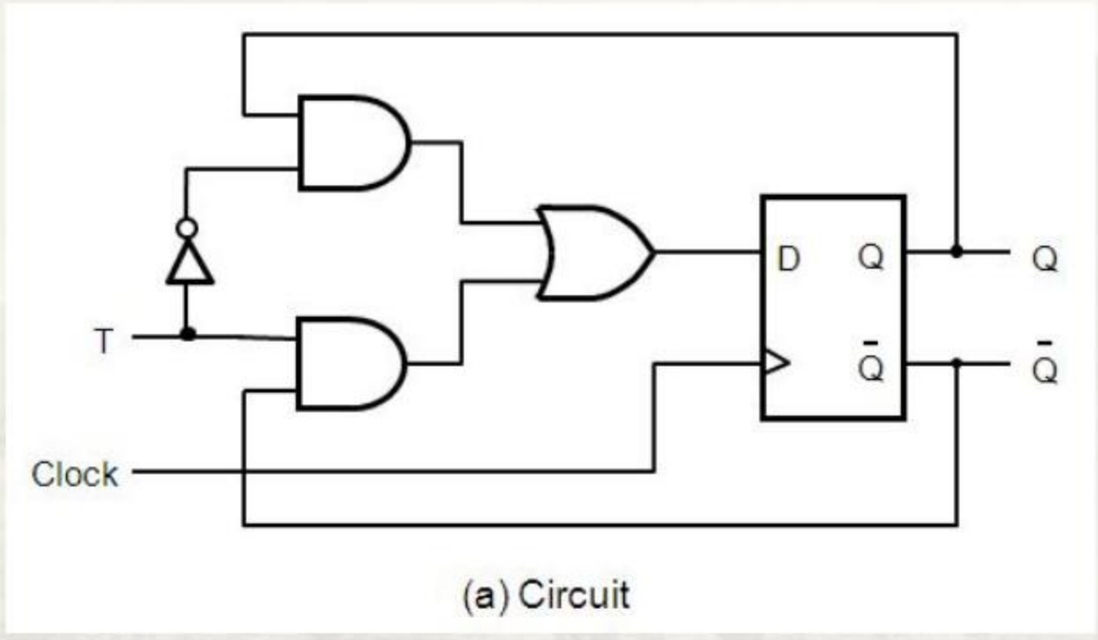


(a) Circuit



(b) Graphical symbol

5.5 T Flip-flop



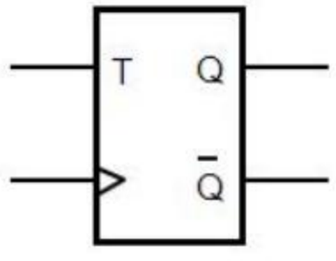
(a) Circuit

T	Q(t + 1)
0	Q(t)
1	$\overline{Q}(t)$

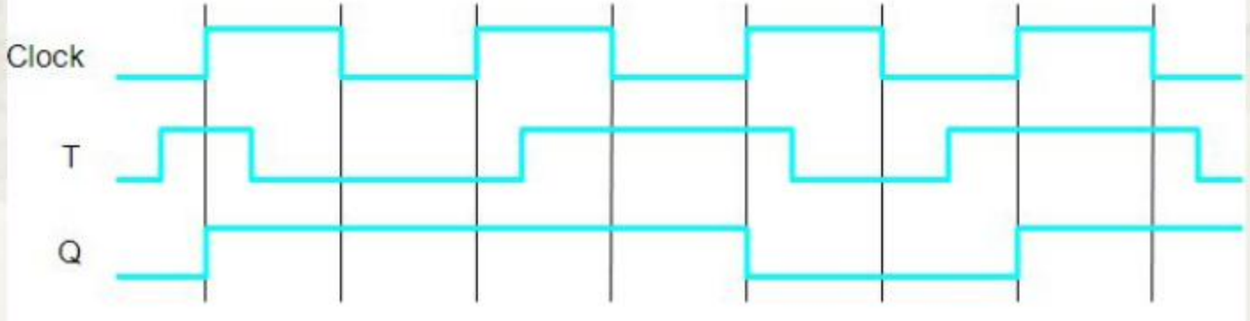
(b) Truth table

$$Q^{n+1} = T\overline{Q^n} + \overline{T}Q^n$$

Toggle flip-flop

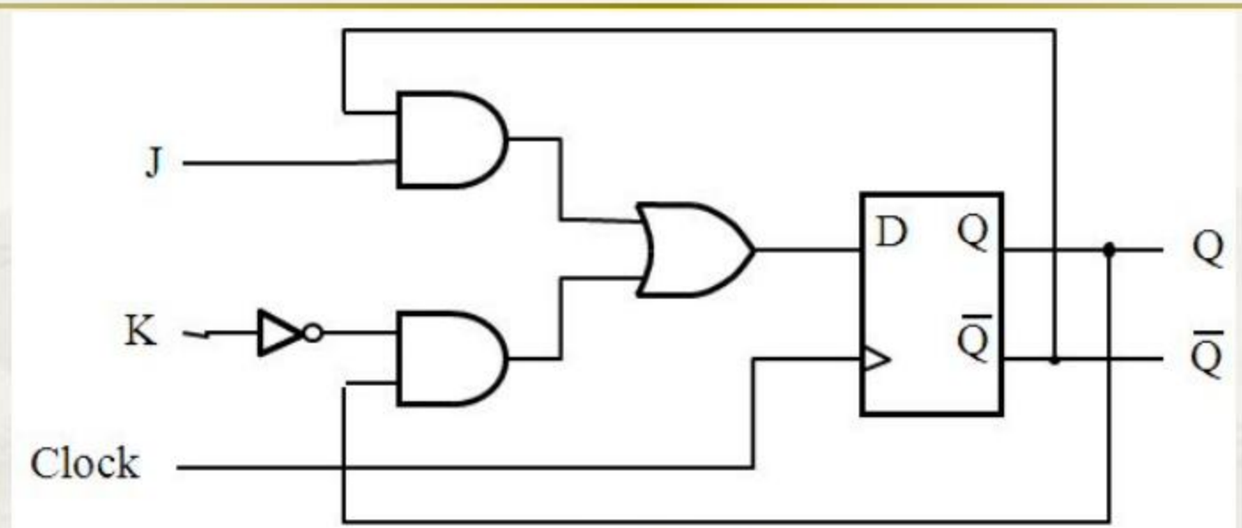


(c) Graphical symbol



(d) Timing diagram

5.6 JK Flip-flop



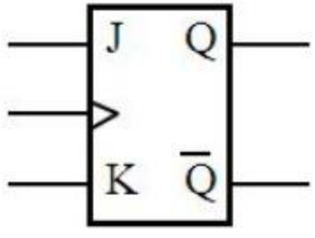
(a) Circuit

J	K	Q(t+1)
0	0	Q(t)
0	1	0
1	0	1
1	1	$\overline{Q}(t)$

(b) Truth table

$$Q^{n+1} = J\overline{Q^n} + \overline{K}Q^n$$

Toggle flip-flop



(c) Graphical symbol

5.7 Synchronous Counters

- Counting
- Timing
- Generating controlling signals(e.g. clock)
- Usually consisting of toggle flip-flops

Understanding Counters' behavior

- A flip-flop stores 1-bit state stably.
- An n-bit(modulo-2^n) counter consists of n flip-flops synchronized by a public clock.
- Counting sequence is formed by simultaneously toggling 1 or more flip-flop in a counter controlled by the public clock.

5.7.1 Counting behavior

- A 3-bit(modulo-8) up-counter

Clock cycle	$Q_2\ Q_1\ Q_0$
0	0 0 0
1	0 0 1
2	0 1 0
3	0 1 1
4	1 0 0
5	1 0 1
6	1 1 0
7	1 1 1
8	0 0 0

Q_1 *changes*

Q_2 *changes*

A single Clk signal

Number of flip-flops?

Types of ff to be used?

E.g.: T(or JK) flip-flops!

5.7.2 Implementation of an n-bit up-counter

- A 3-bit up-counter(modulo-8)

Clock cycle	Q_2 Q_1 Q_0
0	0 0 0
1	0 0 1
2	0 1 0
3	0 1 1
4	1 0 0
5	1 0 1
6	1 1 0
7	1 1 1
8	0 0 0

Q_1 changes

Q_2 changes

How?

Q0 toggles in each Clk cycle

Q1 toggles after Q0=1

Q2 toggles after Q0=1 & Q1=1

And it follows that:

For an n-bit up counter(modulo-2^n)
Q_{n-1} toggles after Q_0~Q_{n-2} are all 1s!

Implement an n-bit up-counter

$Q^{n+1} = T\overline{Q^n} + \overline{T}Q^n$ *when* $T = 1$, *it toggles its state.*

Consider Q_0 *which toggles in every Clk cycle* $\rightarrow \underline{T_0 = 1}$

Consider Q_1 *which toggles if* $Q_0 = 1 \rightarrow \underline{T_1 = Q_0}$

Consider Q_2 *which toggles if* $Q_1 = 1 \,\&\, Q_0 = 1 \rightarrow \underline{T_2 = Q_1 Q_0}$

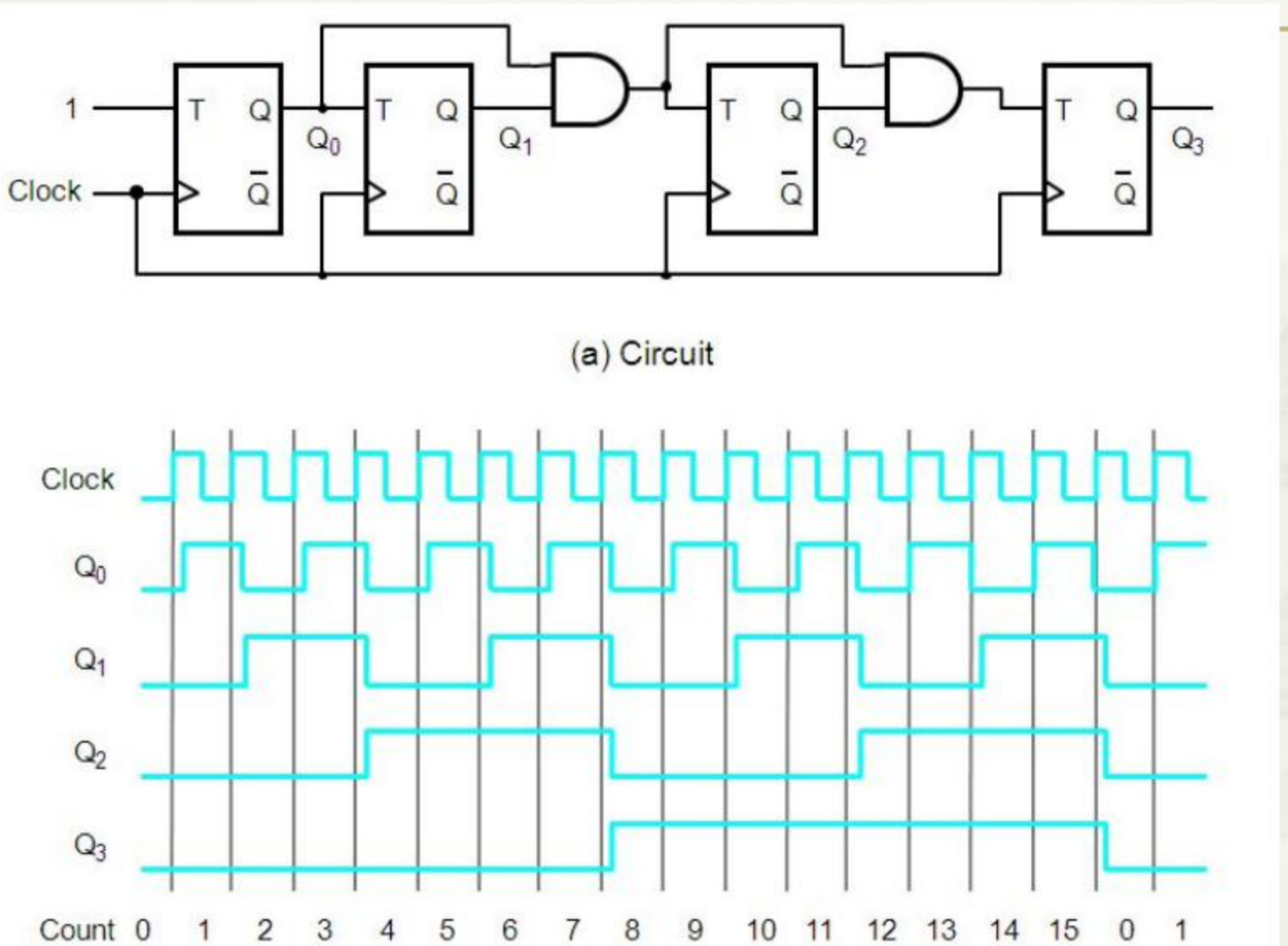
Consider Q_3 *which toggles if* $Q_2 = 1 \,\&\, Q_1 = 1 \,\&\, Q_0 = 1 \rightarrow \underline{T_3 = Q_2 Q_1 Q_0}$

...

and it follows that :

Q_n *toggles if* $Q_{n-1} = 1 \,\&\, \ldots \,\&\, Q_1 = 1 \,\&\, Q_0 = 1 \rightarrow \underline{T_n = Q_{n-1} Q_{n-2} \ldots Q_1}$

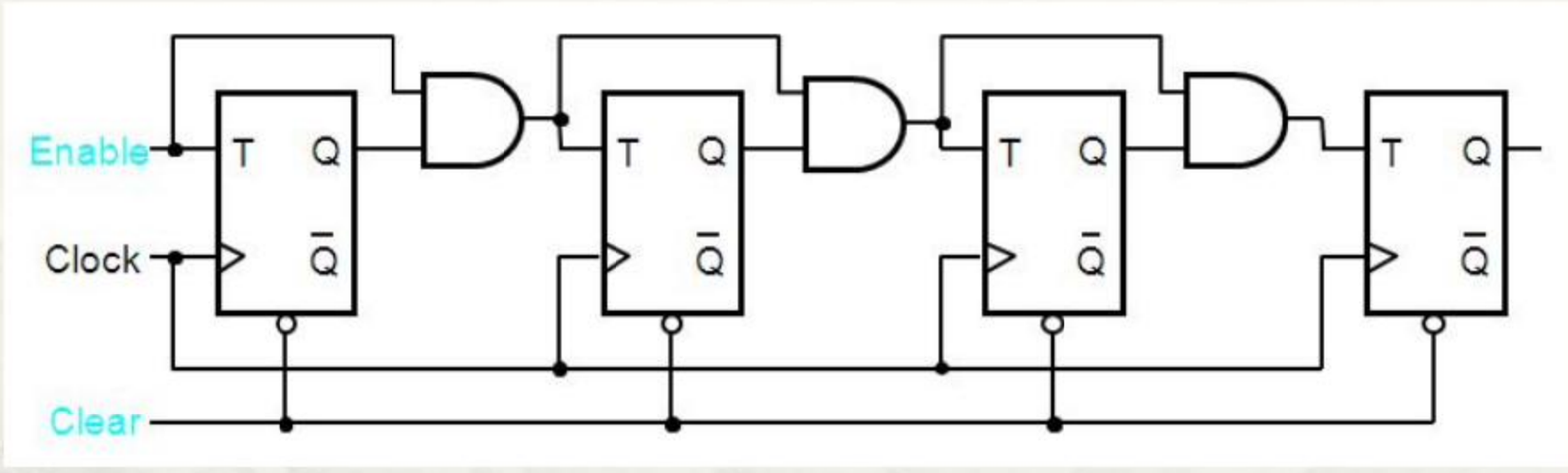
5.7.3 A 4-bit(modulo-16) counter with T flip-flops



(a) Circuit

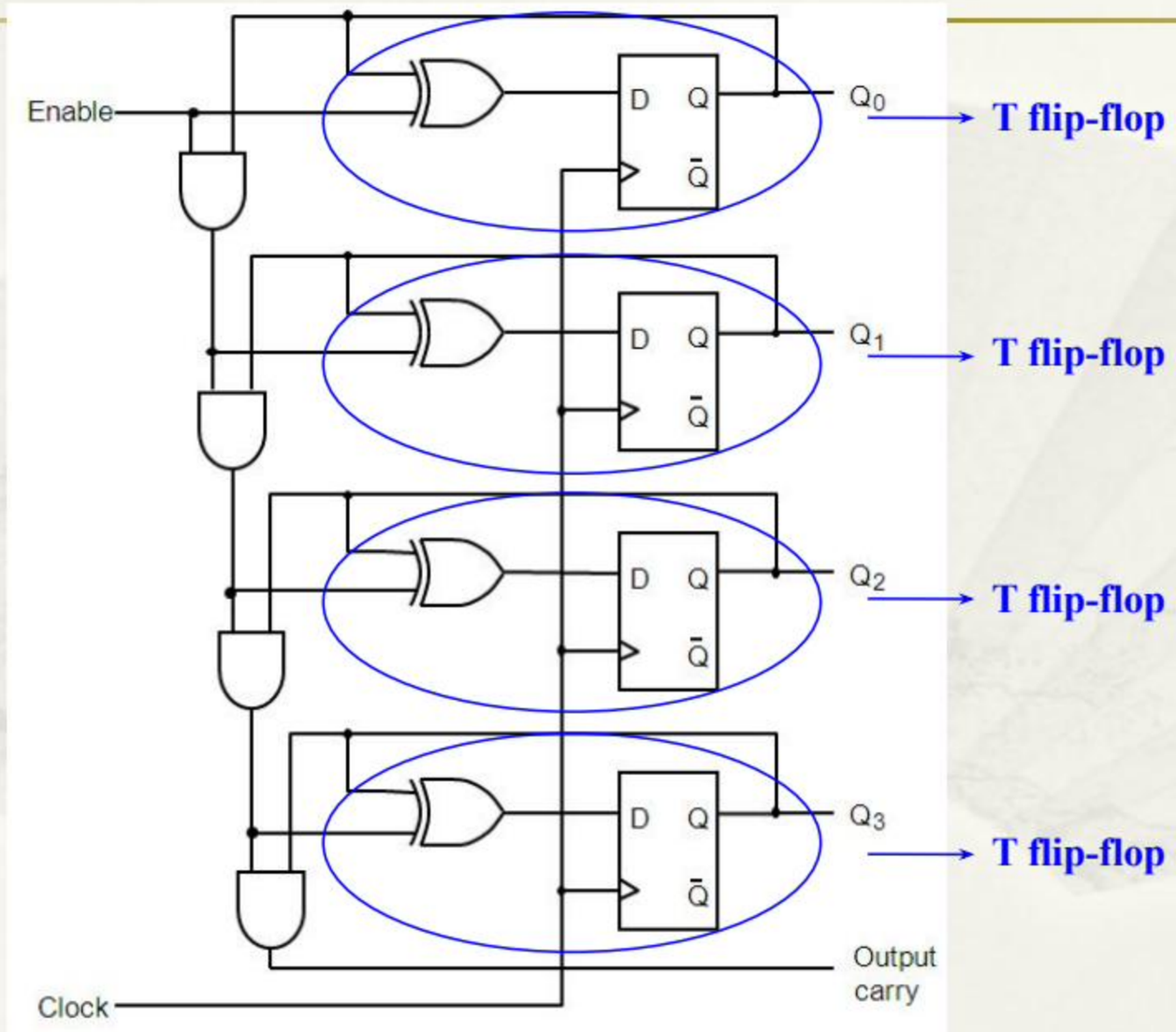
(b) Timing diagram

5.7.4 Enable and Clear Capability



- When Enable=0, the counting process is paused
- When Clear=0(active low), the counter is reset to all 0s
- Enable is synchronized by clock, while Clear is not(Asynchronization)!

5.7.5 A 4-bit counter with D flip-flops



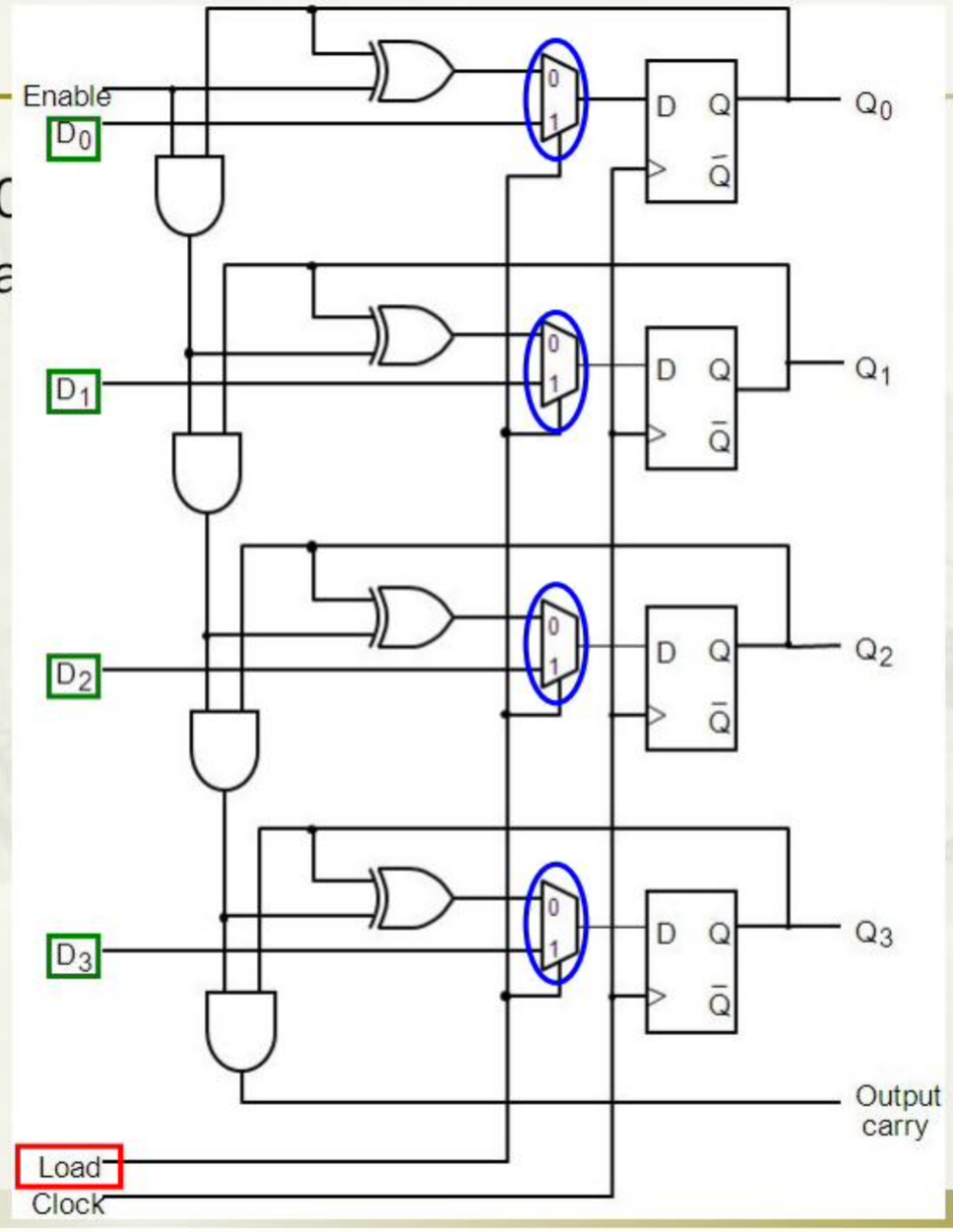
5.7.6 Counters with parallel load

- Unnecessary to count from 0
- Desirable to start with any value to count.

When load=0, the counting process goes on if Enabled.

When load=1, D3~D0 are selected and loaded into the counter when next rising edge of Clock occurs.

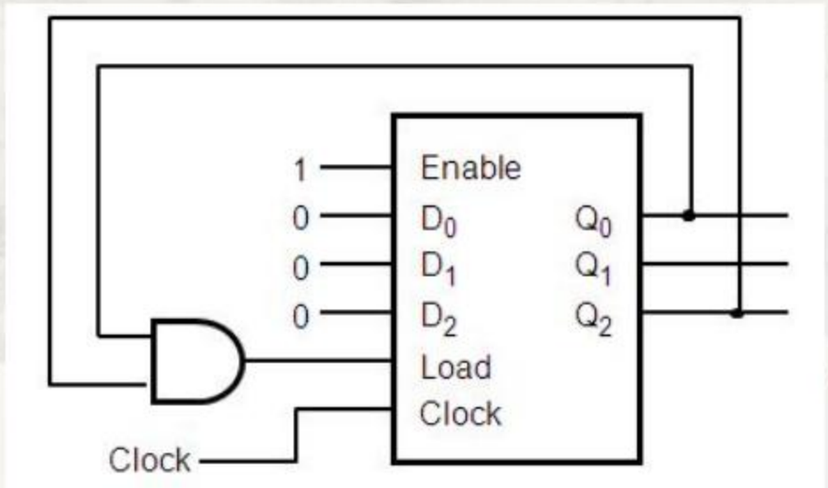
In this case, the counter is reset to $D_3D_2D_1D_0$ and counts from it.



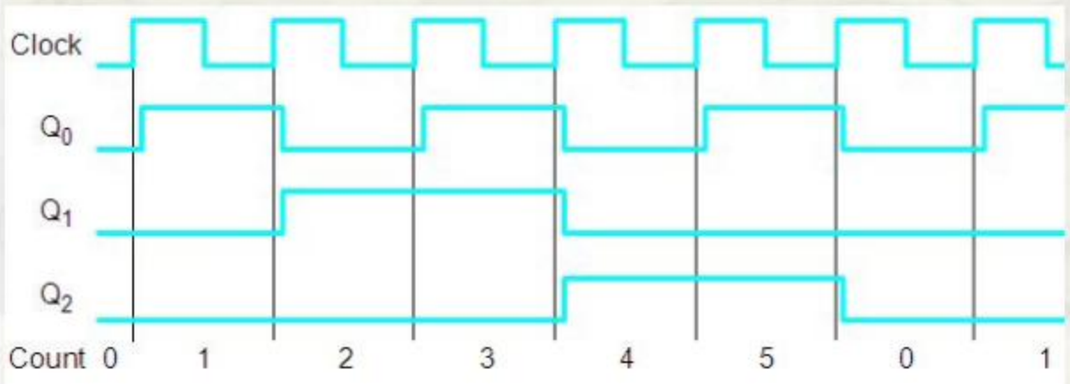
5.7.7 Implementation of any modulo-N counter

- An n-bit up-counter functions as a modulo-2^n.
- Desirable to implement a modulo-N($N<2^n$) counter .
- Skip some(?) counts from the modulo-2^n counting sequence by
  - linking some outputs to the *load* input so as to reset the counter to "$D_{n-1}...D_0$" before it counts to all 1s!

Reset Synchronization!



(a) Circuit



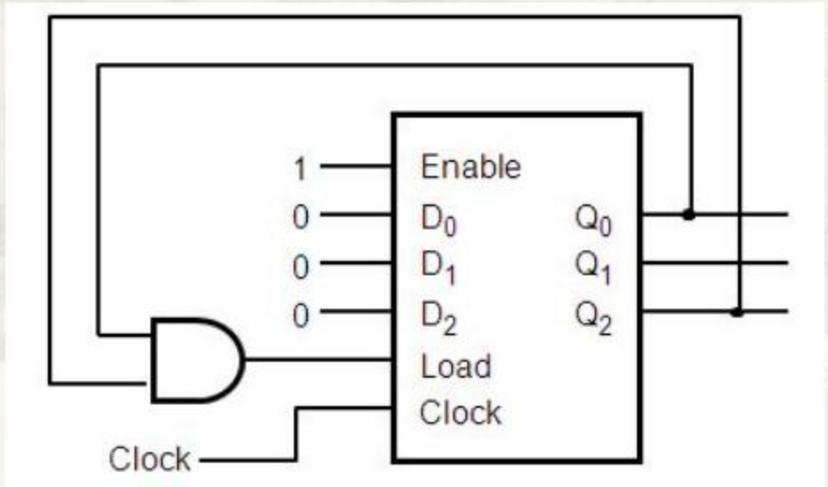
(b) Timing diagram

Modulo-? and other ways out?

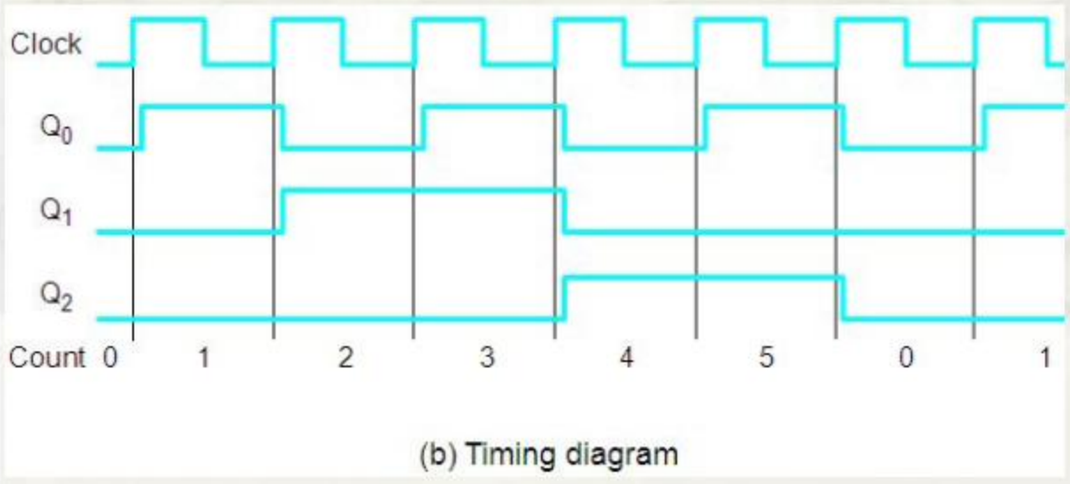
5.7.7 Implementation of any modulo-N counter

- An n-bit up-counter functions as a modulo-2^n.
- Desirable to implement a modulo-N($N<2^n$) counter .
- Skip some(?) counts from the modulo-2^n counting sequence by
  - linking some outputs to the *load* input so as to reset the counter to "$D_{n-1}...D_0$" before it counts to all 1s!

Reset Synchronization!



(a) Circuit

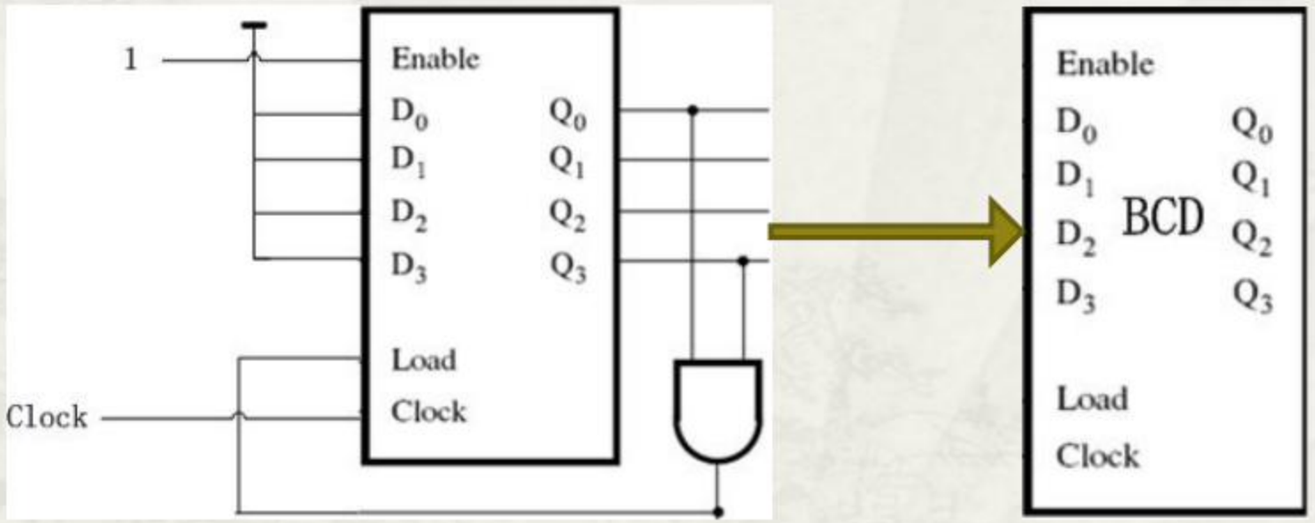


(b) Timing diagram

Modulo-6 counter

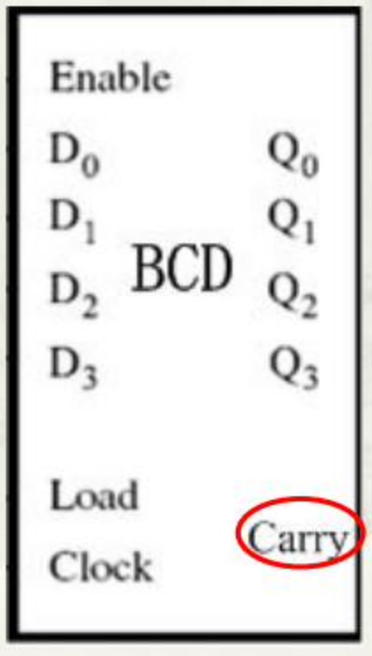
Modulo-10 counter(BCD counter)

- Based on a 4-bit counter(modulo-16) with reset synchronization.

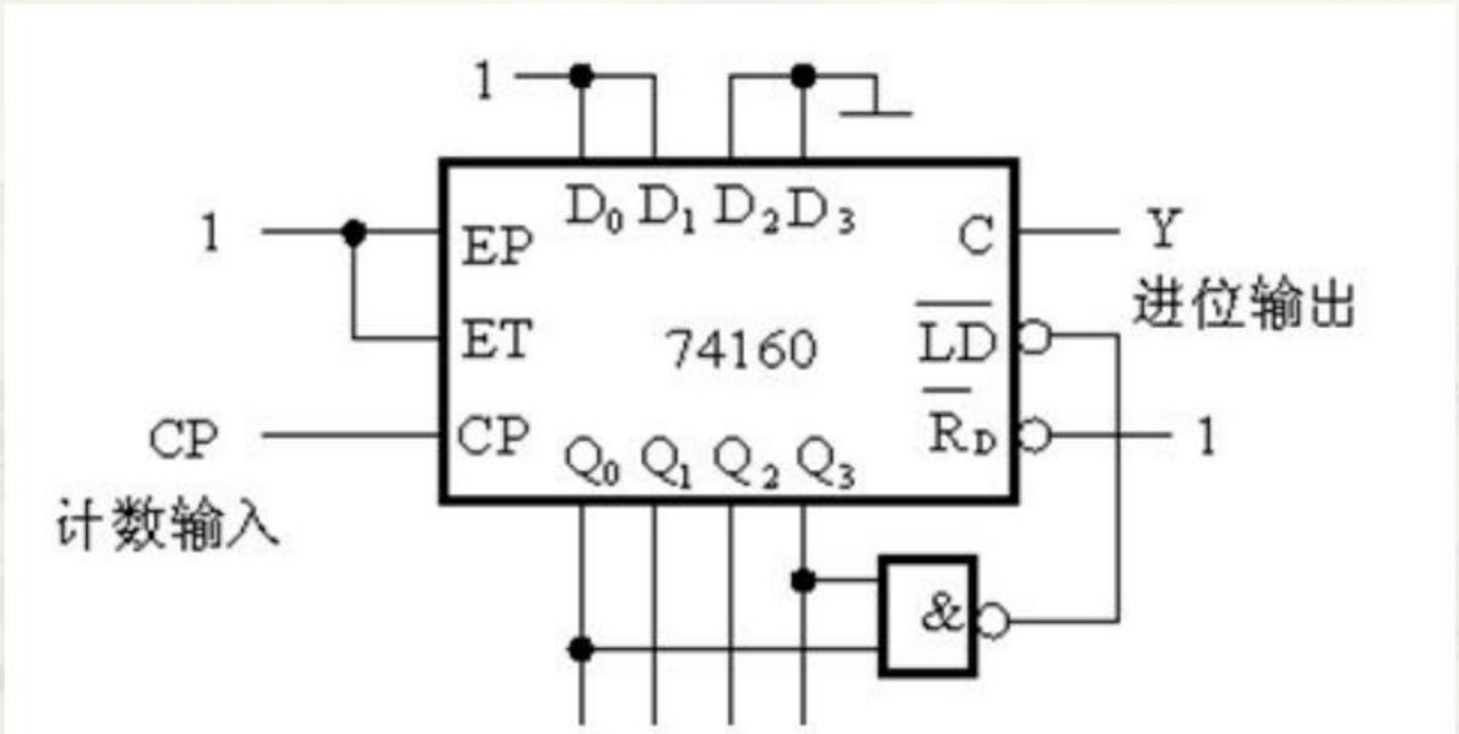


Modulo-10 counter with carry out

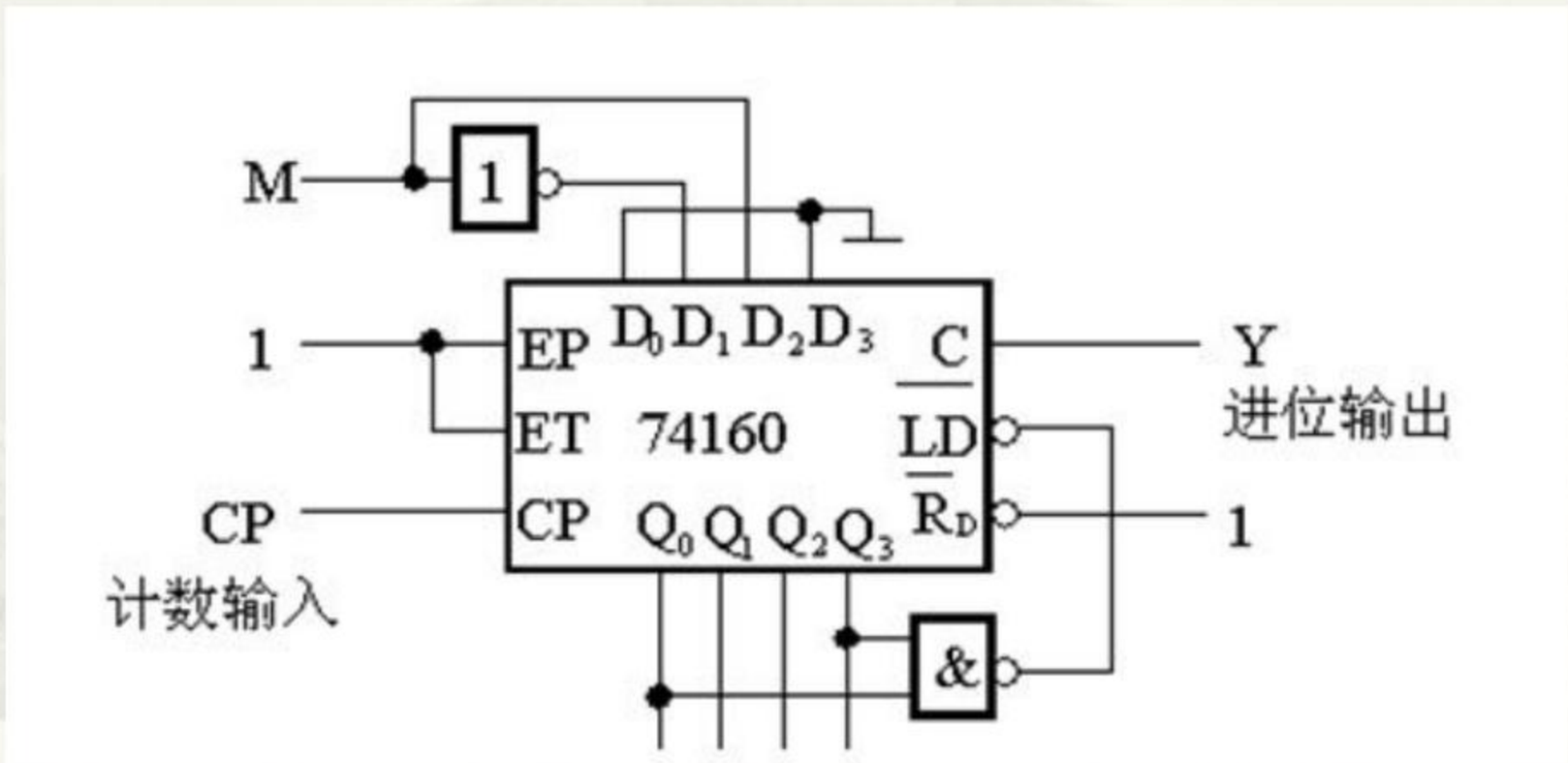
- Used to create a counter modulo a base with larger value.
- Carry=1 when Q=9



Exercise 1:

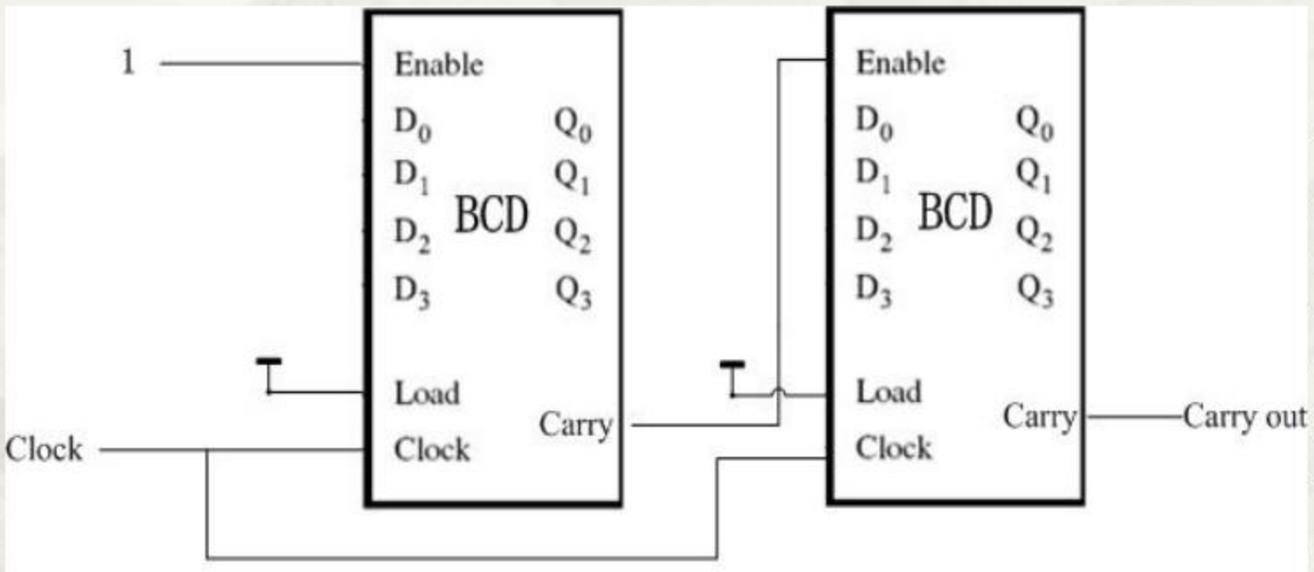

1
1
EP
ET
CP
CP
计数输入
D_0 D_1 D_2 D_3
74160
Q_0 Q_1 Q_2 Q_3
C
Y
进位输出
$\overline{LD}$
$\overline{R_D}$
1
&

Exercise 2:



What if N>2^n ?

- modulo-(N?) counter
  - Cascading 2 BCD counters



Modulo-100 counter

1. Synchronous system

2. Without loading

3. Left counter works in every clock cycle

4. Right one is driven by the Carry of the left one.

5. What's the ratio of counting frequency between the two?

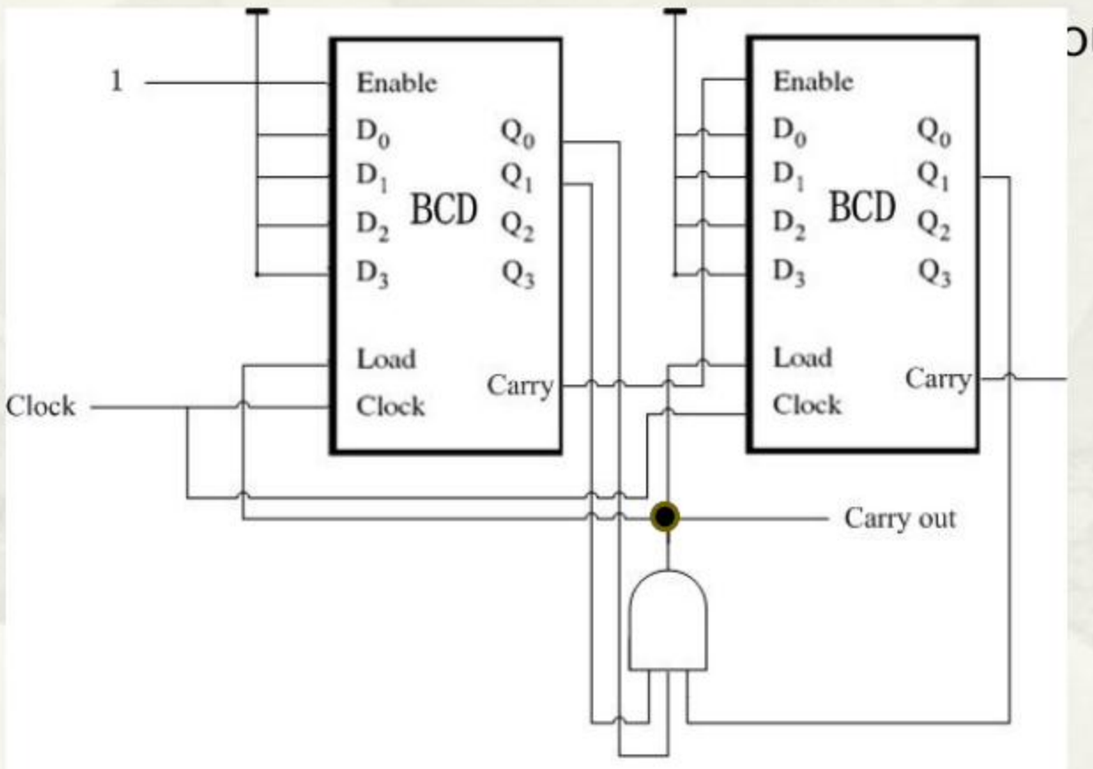
6. The Carry out is generated every ? clock cycles.

7. Range of counts as a whole?

Question: ***Modulo what base if cascading a moludo-A counter and a modulo-B counter? And how about cascading more counters?***

What if N>2^n

- Implementation of a modulo-N counter, N≠A*B
  - Achieved by cascading 2 BCD counters with integral parallel load



Modulo-24 counter

ounter

1. **Synchronous system**

2. **Loads enabled simultaneously.**

3. **Note the inputs of the AND gate!**

4. **Counts from 00~23(2 BCD groups).**

Question: How to implement other modulo-X counter? E.g.: modulo-60?

5.7.8 Modulo-X counters in QUARTUS II

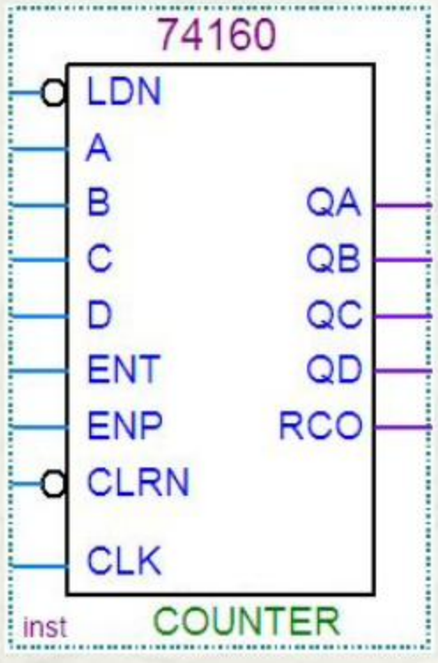
- E.g.: The modulo-10 counter

1. Name: 74160. (74161 as the 4-bit up counter)

2. LDN(Load) is active low.

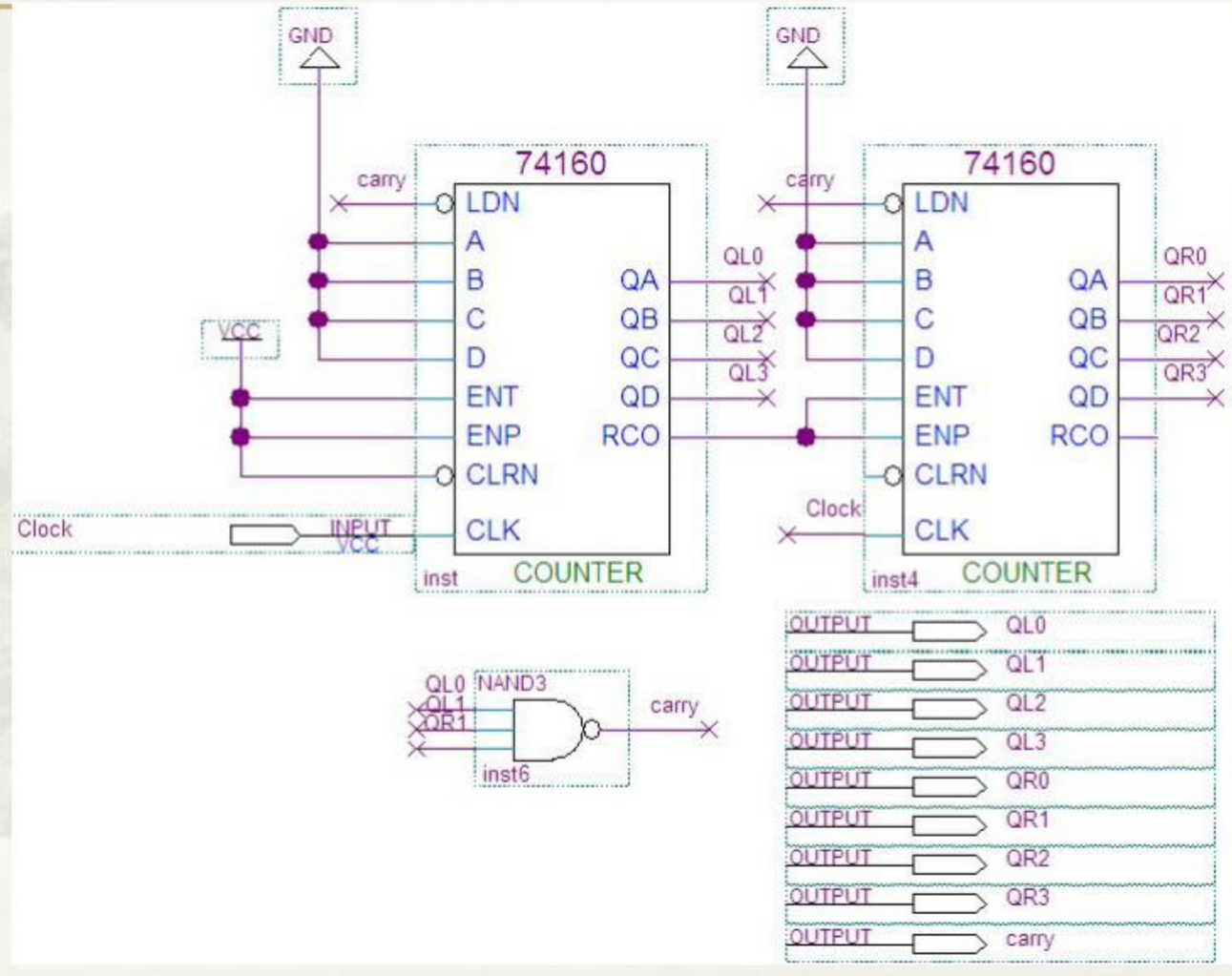
3. ENT & ENP (Enable), RCO(Carry).

4. D~A(D3~D0), QD~QA(Q3~Q0)



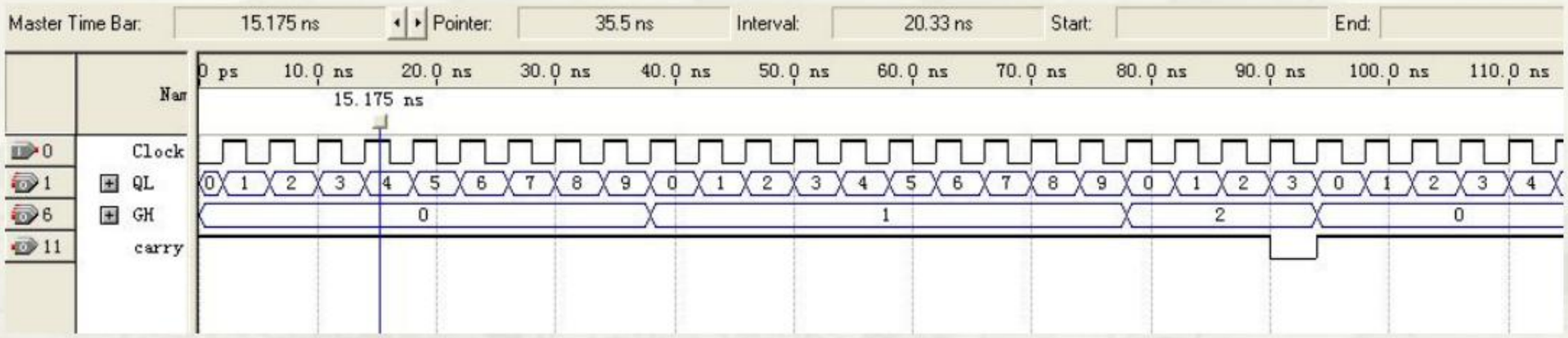
5.7.8 Modulo-X counters in QUARTUS II

- The modulo-24 counter with 74160s

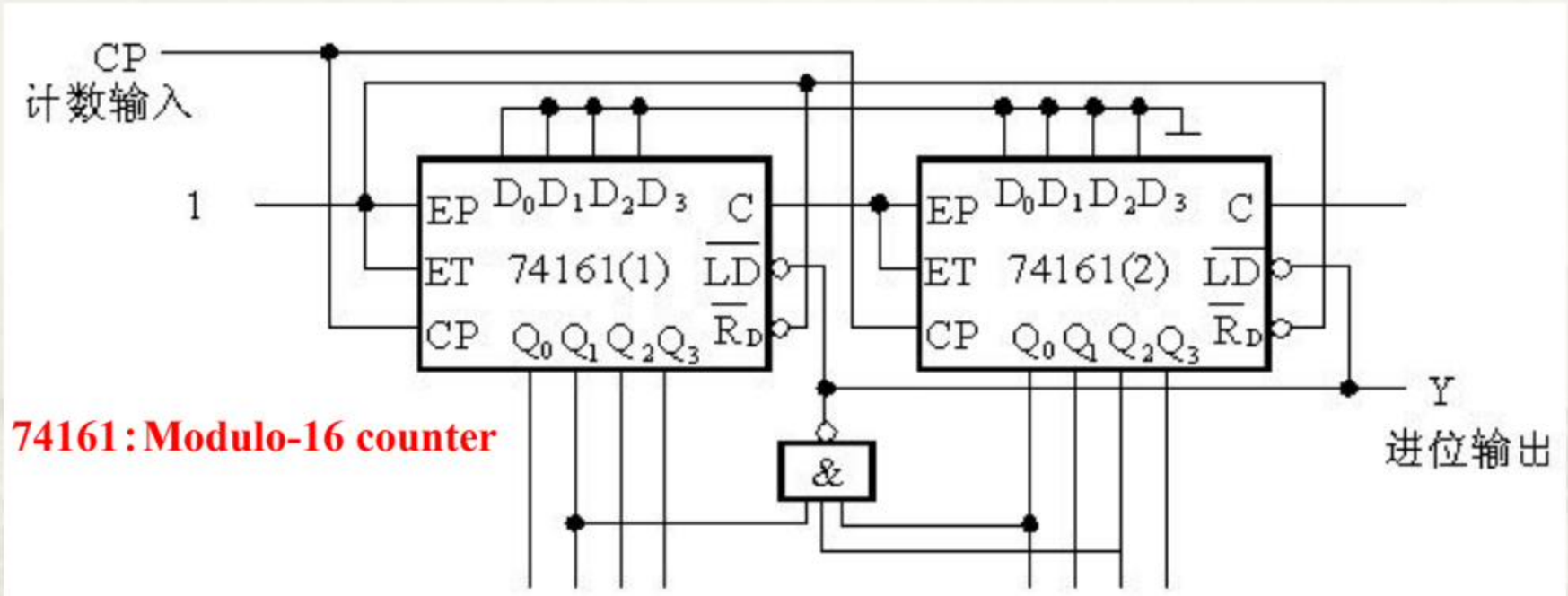


Signal correspondence by name

Simulation result



Exercise 3:



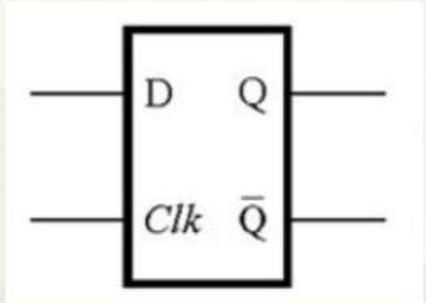
74161 : Modulo-16 counter

What if both counters are replaced with 2 BCD counters?

5.8 Storage elements in VHDL

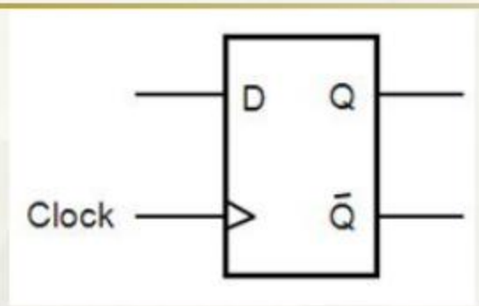
5.8.1 Code for the gated D latch

```
LIBRARY ieee ;
USE ieee.std logic 1164.all ;
ENTITY Dlatch IS                          --D latch as a example
    PORT ( D, Clk : IN STD LOGIC ;
           Q : OUT STD LOGIC) ;
END Dlatch ;
ARCHITECTURE Behavior OF Dlatch IS
BEGIN
    PROCESS ( D, Clk )                    -- D or Clk can cause an output change!
      BEGIN
        IF Clk = '1' THEN                 -- level sensitive
          Q <=D ;
        END IF ;                          -- implied memory
    END PROCESS ;
END Behavior ;
```



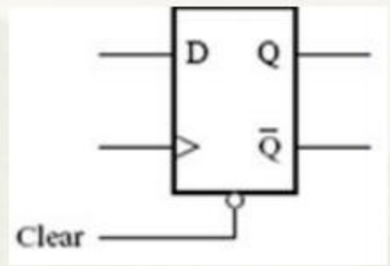
5.8.2 Code for the D flip-flop

```
LIBRARY ieee ;
USE ieee.std logic 1164.all ;
ENTITY D_ff IS
    PORT ( D, Clk : IN STD LOGIC ;    --D flip-flop as a example
           Q : OUT STD LOGIC) ;
END D_ff;
ARCHITECTURE Behavior OF D_ff IS
BEGIN
    PROCESS ( Clk )              --Only active edge can cause an output change
       BEGIN
          IF Clk'EVENT AND Clk ='1'  THEN      --rising edge triggered
             Q <= D ;
          END IF ;
    END PROCESS ;
END Behavior ;
```

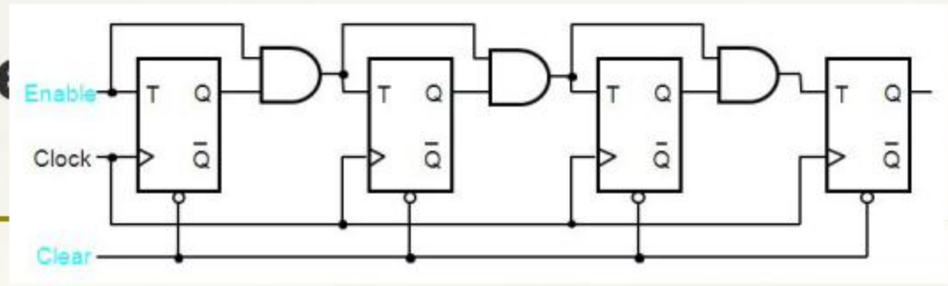


Code for the Dff with asynchronous clear

```
LIBRARY ieee ;
USE ieee.std_logic_1164.all ;
ENTITY DFF IS
    PORT ( D, Clk, Clr : IN STD LOGIC ;
                Q : OUT STD LOGIC) ;
END DFF;
ARCHITECTURE Behavior OF DFF IS
BEGIN
   PROCESS ( Clk, Clr )              --Clear signal NOT clocked
      BEGIN
         IF Clr = '0' THEN          --Clear signal NOT clocked
            Q <= '0';
         ELSIF Clk'EVENT AND Clk ='1'  THEN
            Q <= D ;
         END IF ;
   END PROCESS ;
END Behavior ;
```



5.8.3 Code for a counte




```
LIBRARY ieee ;
USE ieee.std_logic_1164.all ;
USE ieee.std_logic_unsigned.all ;
ENTITY upcount IS                                   --a 4-bit up counter as an example
   PORT ( Clock, Clr, En: IN STD_LOGIC ;            --with clear & enable capability
          Q : OUT STD_LOGIC_VECTOR (3 DOWNTO 0)) ;
END upcount ;
ARCHITECTURE Behavior OF upcount IS
   SIGNAL Count : STD LOGIC VECTOR (3 DOWNTO 0) ;        --internal signal
BEGIN
   PROCESS ( Clock, Clr )
   BEGIN
      IF Clr = '0' THEN                             --asynchronous clear
         Count <= "0000" ;
      ELSIF (Clock'EVENT AND Clock ='1') THEN
         IF En = '1' THEN                           --synchronous enable
            Count <= Count + 1 ;
         END IF ;
      END IF ;
END PROCESS ;
   Q <= Count ;                          -- The Count signal passes the result to Q outside of  the clock process
END Behavior ;
```

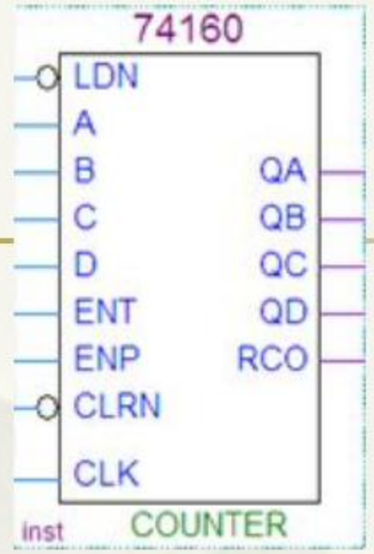
Alternative code

```
LIBRARY ieee ;
USE ieee.std_logic_1164.all ;
USE ieee.std_logic_unsigned.all ;
ENTITY upcount IS
    PORT ( Clock, Clr, En: IN STD_LOGIC ;
                Q : BUFFER STD_LOGIC_VECTOR (3 DOWNTO 0)) ;        -- the BUFFER mode
END upcount ;
ARCHITECTURE Behavior OF upcount IS
BEGIN
    PROCESS ( Clock, Clr )
    BEGIN
        IF Clr = '0' THEN
            Q <= "0000" ;
        ELSIF (Clock'EVENT AND Clock ='1') THEN
            IF En = '1' THEN
                Q <= Q+1 ;                    --Q is directly used as being of the mode BUFFER
            END IF ;
        END IF ;
END PROCESS ;
END Behavior ;
```

Code for a counter with parallel load

```
...
 ENTITY upcount IS
PORT ( Data : IN STD_LOGIC_VECTOR(3 downto 0) ;        --parallel data inputs
          Clock, Clear, Ld : IN STD LOGIC ;
          Q : BUFFER STD_LOGIC_VECTOR(3 downto 0) ) ;
END upcount ;
ARCHITECTURE Behavior OF upcount IS
BEGIN
    PROCESS ( Clock, Clear )
        BEGIN
       IF Clear= '0' THEN
          Q <= "0000" ;
       ELSIF (Clock'EVENT AND Clock = '1') THEN
            IF Ld = '1' THEN                          --parallel load(active high)
               Q <= Data ;
            ELSE
               Q <= Q + 1 ;
            END IF;
       END IF;
    END PROCESS;
END Behavior;
```

Code for a modulo-N counter




```
-- with 74160 counter as the example
LIBRARY ieee;
USE ieee.std_logic_1164.ALL;
USE ieee.std_logic_unsigned.ALL;
entity T74LS160 is
    Port ( Clk : in  std_logic;
           Q : buffer  std_logic_vector(3 downto 0);   --QD,QC,QB,QA
           Carry :   out  std_logic;
           Clr :    in   std_logic;
           Data : in   std_logic_vector(3 downto 0);   --D,C,B,A
           Ld:     in  std_logic;                      --active low
           p :     in   std_logic;                     --ENP
           t :     in    std_logic);                   --ENT
end T74LS160;
```

```
architecture  BEHAVIORAL  of  T74LS160 is
  begin
    Carry<='1' when (Q="1001" and t='1' and Clr='1') else '0';
    process(clk,clr)
      begin
        if Clr='0' then Q<="0000";                --asynchronous clear
        elsif(rising_edge(clk)) then              --the rising_edge function
          if(ld='1') then                         --active low
if(p='1') then
  if(t='1') then
    if(Q="1001")                                  --modulo-10 counter
        Q<="0000";
    else  Q<=Q+1;
    end if;
  else Q<=Q;                                      --state kept if t=0
  end if;
```

```
                else  Q<=Q;              --state kept if p=0
              end if;
            else  Q<=Data;               --parallel load(active low)
            end if;
          end if;
end process;
end BEHAVIORAL;
```